



MIRACLE

★★ 기적처럼 재미있는 Java 프로그래밍 ★★

미라클 자바 프로그래밍

조성호 지음

네트워킹과 소켓 프로그래밍

01 네트워킹 기초 개념

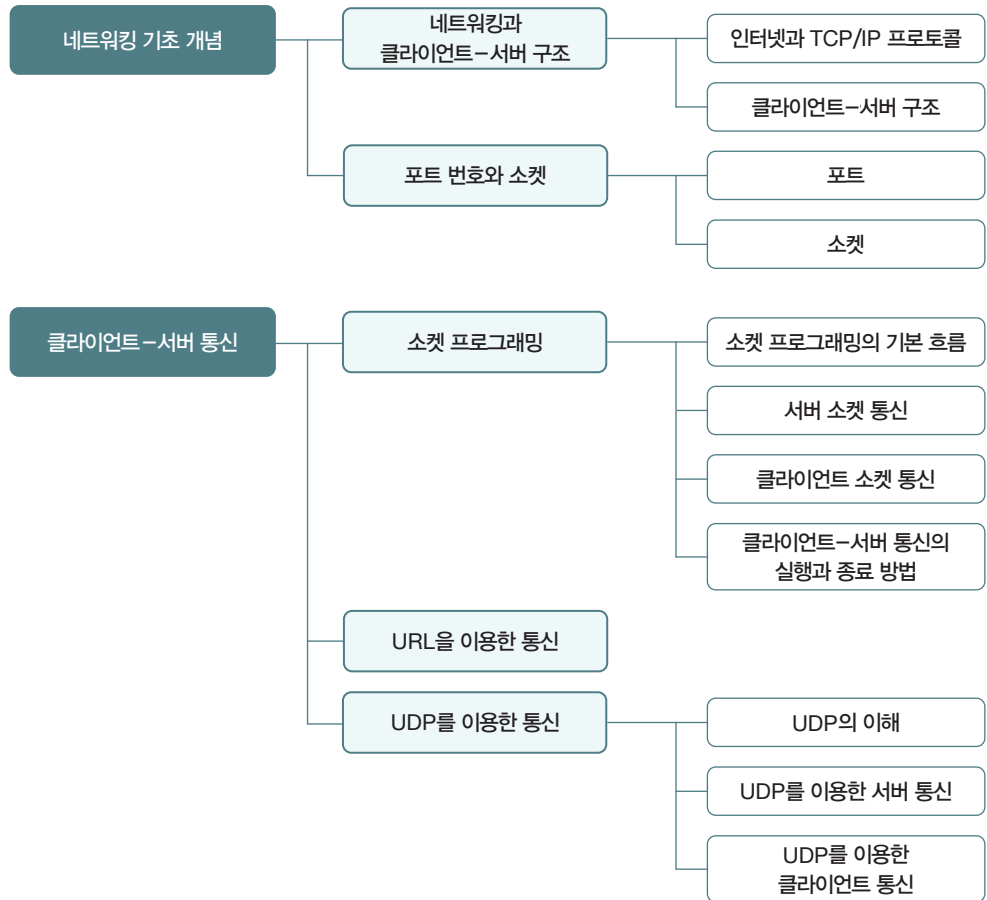
02 클라이언트-서버 통신

요약

연습문제

학습 목표

- 인터넷, TCP/IP, 클라이언트-서버, 포트와 소켓 등 네트워크 관련 용어를 이해한다.
- 소켓 프로그래밍의 기본 흐름을 이해하고, 클라이언트 및 서버 코드를 작성하는 방법을 학습한다.
- URL 및 UDP를 사용하는 네트워크 통신 프로그램의 구조를 살펴본다.



1 네트워크와 클라이언트-서버 구조

1. 인터넷과 TCP/IP 프로토콜

컴퓨터 네트워크(computer network)란 두 대 이상의 컴퓨터가 데이터를 주고받는 통신 시스템을 의미한다. 컴퓨터를 전선으로 연결하고, 이 선에 데이터를 보냄으로써 최초의 데이터 통신이 탄생했다. 데이터 통신 초창기에는 2대의 컴퓨터를 일대일로 연결했지만, 기술의 발전으로 여러 대의 컴퓨터가 서로 데이터를 주고받을 수 있게 되었다. 이렇게 컴퓨터 여러 대가 데이터를 주고받는 통신 시스템이 컴퓨터 네트워크이며, 줄여서 네트워크라 부른다.

단순히 컴퓨터를 선으로 연결하는 것만으로는 통신이 이루어지지 않는다. 통신이 원만하게 이루어지려면 통신 당사자 간 약속을 해야 한다. 예를 들어, 누가 먼저 데이터를 보낼지, 얼마만큼 빠른 속도로 데이터를 보낼지를 정해야 한다.

종이컵 전화의 통신 방식을 생각해보자. 두 사람 사이를 종이컵과 선으로 연결한 뒤, 동시에 컵을 입이나 귀에 대면 통신이 이루어지지 않는다. 한 사람이 말을 할 때는 다른 한 사람이 컵을 귀에 대고 있어야 통신을 할 수 있다. 이렇듯 통신을 위한 약속을 프로토콜(protocol)이라 부르며, 우리 말로는 통신 규약이다. 컴퓨터 네트워크는 유선 혹은 무선의 연결과 프로토콜로 구성된다.



그림 15-1 네트워크와 프로토콜

초기의 컴퓨터 통신은 가까운 거리를 연결하는 수준이었다. 한 건물 내에 있는 컴퓨터 또는 길어야 수백 미터 정도 떨어진 컴퓨터만 연결할 수 있었다. 이렇게 가까운 거리에 연결된 네트워크를 **LAN**(Local Area Network)이라 부른다. LAN이란 수 미터에서 약 1킬로미터 내의 컴퓨터를 연결하는 **근거리 네트워크**를 의미한다.

초기의 LAN은 일정한 표준이 없었기 때문에 여러 회사에서 각자의 규격을 가진 LAN을 출시하였다. 따라서 서로 다른 LAN들 사이에 호환성이 없어 네트워크 간 통신이 원활하지 않았다. 특히 미국의 국방성은 이러한 LAN의 호환성 문제로 큰 어려움을 겪고 있었다. 국방에서 가장 중요한 요소는 일관된 정보 전달이지만, 육군, 해군, 공군이 서로 다른 종류의 LAN을 사용하면서 데이터를 한 번에 전송할 수 없었다.

이러한 문제를 해결하기 위해 1960년대 미국 국방성은 서로 호환되지 않는 LAN을 묶어 하나의 네트워크로 만들기 위한 연구를 진행하였는데, 이 프로젝트가 ‘Advanced Research Project Agency ARPA’이다. 이 프로젝트의 결과로 만들어진 네트워크가 1969년 만들어진 아파넷(ARPAnet)이며, 이 아파넷이 전 세계에 공개되어 지금의 인터넷(internet)이 되었다.

그러나, 모든 LAN을 선으로 연결한다고 해서 인터넷이 되는 것은 아니기 때문에 서로 다른 LAN에서 데이터를 어떻게 주고받을 것인가에 대한 프로토콜이 필요했다. 새로 만들어진 프로토콜은 서로 다른 종류의 LAN을 넘어 다니면서 목적지에 데이터를 전송해야 했다. 이러한 환경을 반영하여 **네트워크와 네트워크를 넘어 목적지까지 데이터를 전송하는 프로토콜인 IP**(Internetworking Protocol)가 탄생했다. 인터넷을 사용한다는 것은 곧 IP를 사용하여 데이터를 전송한다는 의미이다.

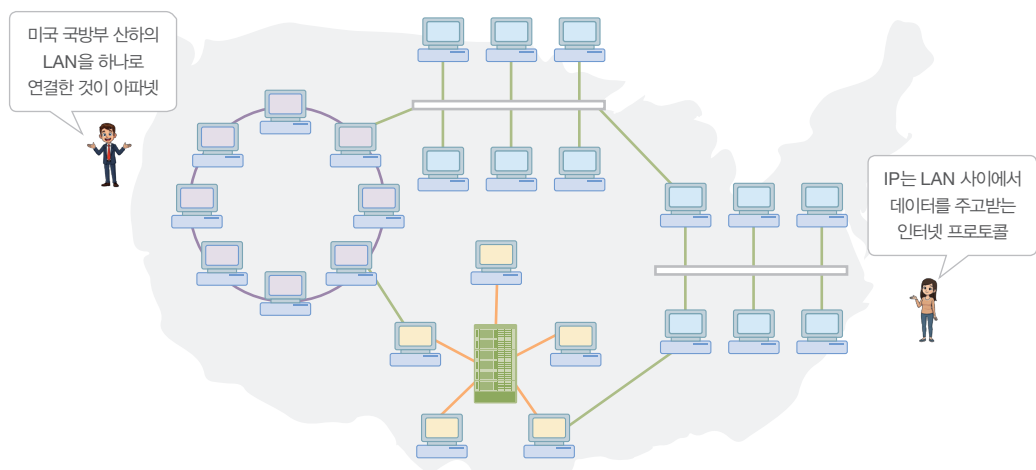


그림 15-2 인터넷과 TCP/IP 프로토콜

인터넷 프로토콜^{IP}을 처음 만들 당시, 목적지까지 데이터를 보내는 데는 성공했으나 데이터가 사라지거나, 순서가 뒤바뀌거나, 일부가 훼손되는 일이 발생하였다. 이를 보완하기 위해 새로운 프로토콜을 만들었는데, 이 프로토콜은 **보내진 데이터가 원래의 상태로 도착하는지를 감시하는 역할**을 한다. 또한 다양한 응용 프로그램으로부터 전달받은 데이터를 목적지의 프로그램에 전달하는 **창구 역할**도 한다. 이러한 역할을 담당하는 프로토콜을 **TCP** Transmission Control Protocol라고 하며, 인터넷을 사용하기 위해서는 TCP/IP를 사용해야 한다.

TCP/IP를 통해 데이터가 전송되는 과정은 택배 배송 과정과 닮았다. TCP는 택배 접수처, IP는 배송 트럭이라 생각하면 된다. IP는 여러 LAN을 통과하여 배송지까지 데이터를 배달하는 일만으로도 빠듯하다. TCP는 이러한 IP를 도와 데이터를 접수하고, 빠진 데이터가 없는지 확인한다.

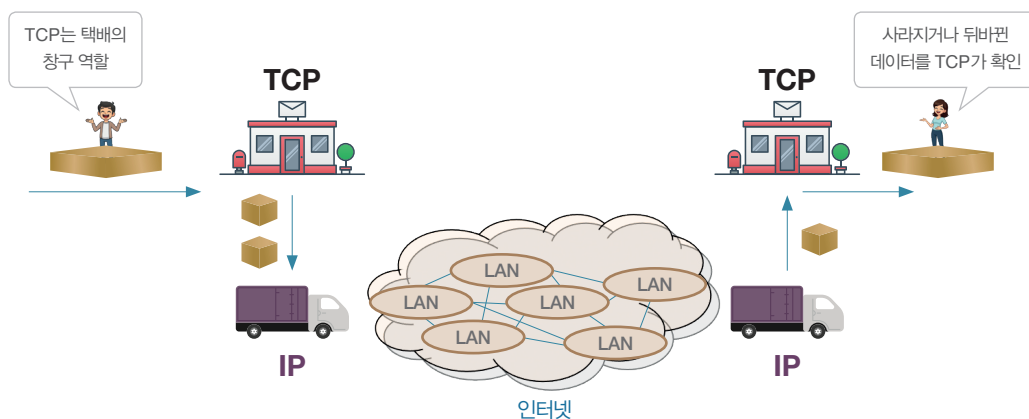


그림 15-3 TCP와 IP의 역할

TCP의 복잡한 작업이 불필요한 경우도 있다. 예를 들어 유튜브에서 동영상을 보고 있다고 가정해보자. 동영상을 끊김 없이 재생하려면 데이터가 빨리 전송되어야 한다. 또한 동영상 데이터의 경우 전체의 데이터 중에서 일부의 데이터가 빠지거나 혹은 순서가 변경되어도 전체 흐름에는 큰 지장이 없다. 이러한 상황에서는 TCP의 작업이 오히려 부담이 되므로, 이를 보완하기 위해 만들어진 프로토콜인 **UDP** User Datagram Protocol를 사용하는 것이 적합하다. UDP는 IP에서 올라온 패킷을 특별한 작업 없이 응용 프로그램에 전달하기 때문에 빠르게 작동하는 대신 전송 오류를 처리하지 않는다.

2. 클라이언트-서버 구조

우리가 사용하는 웹 시스템은 클라이언-서버^{client-server} 구조를 기반으로 한다. **클라이언트란 서비스를 요청하는 컴퓨터이다.** 우리가 사용하는 웹 브라우저는 클라이언트 소프트웨어를 대표하는 단어이며, 제품명으로는 익스플로러, 크롬, 파이어폭스가 있다.

웹 브라우저(클라이언트)는 서버 컴퓨터에 접속하여 웹페이지를 요청하는데, 이때 사용하는 프로토콜은 하이퍼텍스트(웹페이지)를 전송하기 위한 프로토콜이다. 정식 명칭은 Hyper Text Transfer Protocol이며, 약자로 HTTP라 쓴다. 서버가 요청을 받으면, HTML(Hyper Text Markup Language)을 사용하여 만들어진 웹페이지를 클라이언트에 전송한다. 이렇게 전달받은 웹페이지를 웹 브라우저가 화면에 보여준다.

클라이언트 소프트웨어는 사용자의 필요에 따라 종료시킬 수 있지만, 서버 소프트웨어는 다르다. 클라이언트가 언제 서비스를 요청할지 모르기 때문에 항상 대기 상태로 기다려야 한다. 이렇게 종료되지 않은 채 서비스를 계속 진행하는 프로그램을 **데몬**(daemon)이라 부르며, 웹 시스템의 서버 쪽에는 HTTP(웹 서비스)를 받아 줄 데몬이 설치되어 있어야 한다. 데몬은 프로토콜 이름 뒤에 D를 붙이기 때문에, 웹 데몬을 HTTPD(Hyper Text Transfer Protocol Daemon)라 부른다. HTTPD의 제품명으로는 아파치(Apache), 톰캣(Tomcat), IIS(Internet Information Service) 등이 있다. **서버는 데몬이 설치된 컴퓨터를 말한다.**

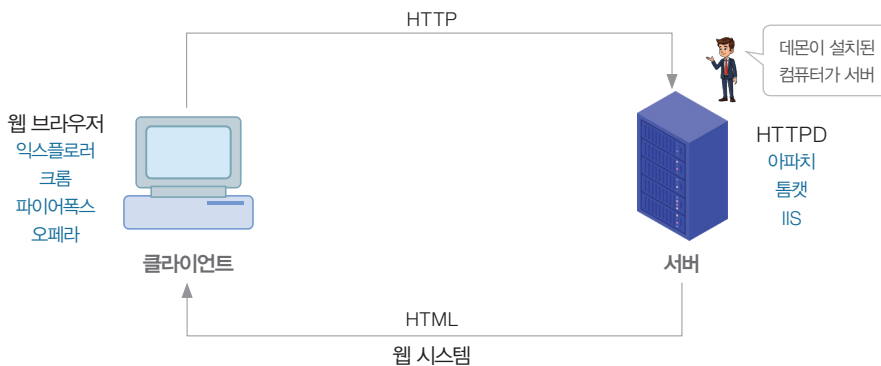


그림 15-4 웹 시스템과 클라이언트-서버 구조

TIP 여기서 HTTPD가 설치된 컴퓨터를 웹 서버라고 부르며, 메일 데몬인 SMTPD(Simple Mail Transfer Protocol Daemon)가 설치된 컴퓨터를 메일 서버라 부른다. 파일 전송 데몬인 FTPD(File Transfer Protocol Daemon)가 설치된 컴퓨터는 파일 서버라 부른다.

2 포트 번호와 소켓

1. 포트

인터넷에 연결된 각 컴퓨터를 구분할 때 사용하는 주소를 IP 주소라 부른다. IP 주소는 인터넷상에서 특정 컴퓨터를 찾아가는 데 사용되며, 이는 아파트의 동 번호에 비유할 수 있다. 그러나 IP 주소만으로는 통신 목적지를 정확히 지정할 수 없다. 이를 이해하기 위해 ‘IPark 아파트 203동’의 어느 집에서 중국집에 짜장면을 주문하고, 같은 시각 다른 집에서 짬뽕을 주문했다고 가정해보

자. 만약, 두 집 모두 자신의 방 번호(호수)를 말하지 않았다면, 배달원은 어느 집으로 음식을 배달해야 할지 알 수 없게 된다.



그림 15-5 중국집 진짜루의 음식 배달

이와 마찬가지로, 컴퓨터에는 네트워크를 사용하는 여러 개의 프로그램이 있으며, 하나의 컴퓨터에서 카카오톡과 크롬 브라우저를 같이 실행시키면 이들의 IP 주소는 같다. 따라서 데이터를 카카오톡에 보낼지, 크롬 브라우저에 보낼지를 결정하기 위해서는 아파트의 호수 역할을 하는 주소가 필요하다. 이 역할을 하는 주소를 포트_{port}라 부르며, 이는 TCP가 응용 프로그램을 구분하기 위해 사용하는 주소 체계이다.

포트라는 용어는 원래 항구에서 배의 화물을 싣거나 내리는 장소를 의미한다. 항구에는 여러 개의 포트(선착장)가 있으며, 배가 들어오면 항구 관리자로부터 사용할 수 있는 빈 포트를 할당받는다. 해당 선박은 할당받은 포트에 정박하여 작업을 진행하는데, 컴퓨터도 이와 비슷하게 동작한다. 컴퓨터의 포트 번호 크기는 16비트이며, 운영체제는 0에서 65535 사이의 빈 포트 번호 중 하나를 네트워크를 사용하려는 프로그램에 할당한다. 해당 프로그램은 할당받은 포트를 사용하여 원격지 프로그램과 데이터를 주고받는다. 이처럼 포트는 같은 컴퓨터 내에서 실행 중인 여러 프로그램을 구분하는 주소이다.

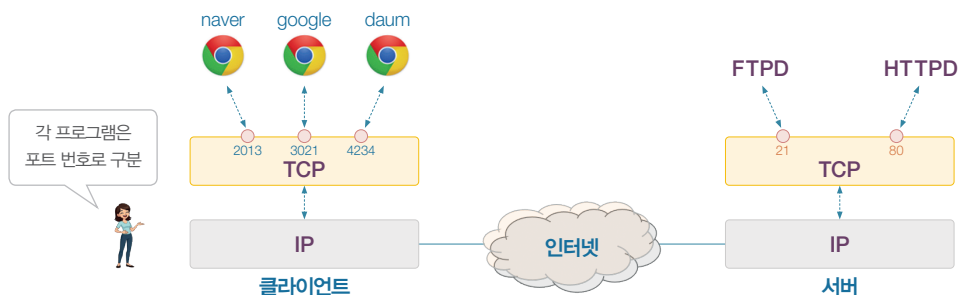


그림 15-6 포트 번호

Note**잘 알려진 포트**

네트워크 통신에서 포트 번호는 0부터 65535까지의 범위를 가지며, 클라이언트와 서버 프로세스는 이 포트 번호를 사용하여 통신한다. 일반적으로 클라이언트가 사용하는 포트 번호는 운영체제가 자동으로 부여하지만, 서버 프로세스의 포트 번호는 클라이언트가 미리 알고 있어야 접속할 수 있다. 웹 서버나 이메일 서버와 같은 주요 서비스의 포트 번호가 실행될 때마다 바뀐다면, 클라이언트는 서버의 위치를 파악할 수 없으므로 통신할 수 없게 된다. 이러한 문제를 보완하고 통신의 표준화를 위해, 0번부터 1023번까지의 특정 포트 번호는 특정 서버 프로세스에 미리 할당했는데, 이를 잘 알려진 포트(Well-known Ports) 번호라 부른다. 예를 들어, 웹 서버(HTTP)의 포트 번호는 80번으로 고정되어 있다.

2. 소켓

네이버는 하루에도 수만 명의 사용자가 접속한다. 이들은 모두 대표 주소(<http://naver.com>)를 통해 방문하며, 이때 포트 80번을 사용한다. 네이버 서버 입장에서 보면 80번 포트에 수만 명의 사용자가 몰려오는 것과 같기 때문에, 하나의 포트에 수많은 사용자에게 어떻게 서비스를 제공할 수 있을 것인가의 문제가 발생한다. 이를 220V 콘센트가 하나뿐인 상황에 비유할 수 있다. 노트북, 스마트폰, 블루투스 이어폰을 동시에 충전해야 한다면, 멀티탭을 사용하여 해결할 수 있다. 포트도 마찬가지이다. 서버는 80번 포트를 열어 둔 상태에서, 마치 그 포트에 멀티탭을 연결하듯 각 사용자에게 개별적인 연결 공간을 나눠주면 된다. 이때 사용자가 접속하는 하나의 콘센트를 소켓(socket)이라 부른다.

소켓은 같은 포트에 연결되어 여러 통신 작업을 동시에 처리할 수 있는 소프트웨어적인 접속 장치이다. 네트워크에 있어서 응용 프로그램이 접속하는 포인트는 소켓이다. 그래서 네트워크를 이용한 프로그래밍을 소켓 프로그래밍이라 부른다.

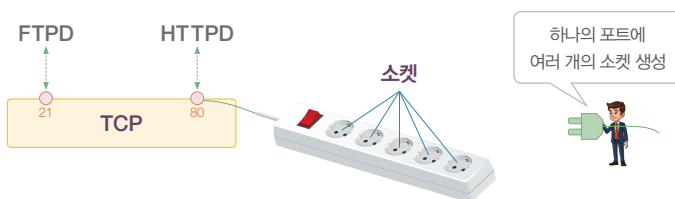


그림 15-7 포트와 소켓

생성형 AI와 공부하기

컴퓨터 네트워크에서 인터넷, 클라이언트-서버, 데몬, IP, TCP, UDP, 포트, 소켓이 서로 어떻게 연결되는지 알기 쉽게 설명해줘.

1 소켓 프로그래밍

1. 소켓 프로그래밍의 기본 흐름

소켓은 파일과 마찬가지로 `sd = Socket()`을 호출하면 소켓 기술자 `socket descriptor`를 반환한다. 이후 모든 작업은 소켓 기술자(`sd`)를 통해 이루어지며, 소켓 기술자는 운영체제가 소켓을 식별하는 데 사용하는 정숫값이다.

[그림 15-8]은 클라이언트와 서버 소켓 간의 통신 절차를 보여준다. 파일 입출력과 비교하면 `sd = socket()` 이후에 클라이언트가 서버에 연결을 요청하는 `connect()`와 클라이언트의 요청을 수락하는 서버의 `accept()`가 추가된다. 네트워크 통신을 위해선 서버 프로세스가 클라이언트보다 먼저 실행되어 클라이언트의 연결을 기다려야 한다.

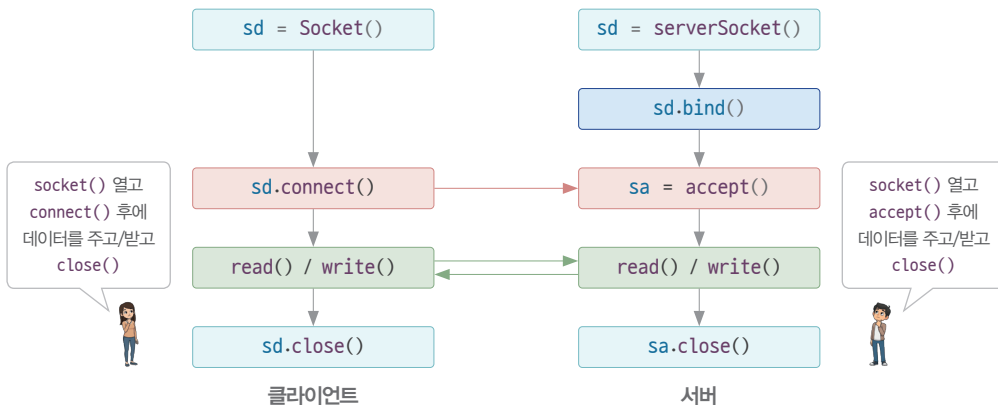


그림 15-8 클라이언트 소켓과 서버 소켓의 통신 절차

서버는 다음과 같은 순서로 작동한다.

- ① **serverSocket():** 통신을 위한 소켓을 생성하고 소켓 기술자를 얻는다.
- ② **bind():** 생성된 소켓에 IP 주소와 포트 번호를 할당하여 소켓이 특정 주소와 포트로 들어오는 연결을 수신할 수 있도록 한다.

- ③ **accept()**: 클라이언트의 연결 요청이 들어오면 이를 수락하고, 실제 데이터 통신에 사용할 새로운 통신 소켓을 생성하여 클라이언트와 1:1 연결을 설정한다. 반환값은 새로운 소켓 기술자인 `sa`다.
- ④ **read()/write()**: 클라이언트와 데이터 송수신을 수행한다. 자바에서는 `getInputStream()`과 `getOutputStream()` 메서드를 통한 스트림을 사용하여 데이터를 전송한다.
- ⑤ **close()**: 통신을 마친 후 소켓 기술자를 닫아 연결을 종료하고 자원을 해제한다.

클라이언트가 소켓을 이용해 서버와 통신하는 기본 절차는 다음과 같다.

- ① **socket()**: 통신을 위한 소켓을 생성하고 소켓 기술자(`sd`)를 얻는다.
- ② **connect()**: 서버의 IP 주소와 포트 번호를 사용하여 서버에 연결을 요청한다. 서버의 `accept()`가 이 요청을 수락하면 연결이 설정된다.
- ③ **write()/read()**: 서버와 데이터 송수신을 수행한다. 자바에서는 `getInputStream()`과 `getOutputStream()` 메서드를 통한 스트림을 사용하여 데이터를 전송한다.
- ④ **close()**: 통신을 마친 후 소켓 기술자를 닫아 연결을 종료하고 자원을 해제한다.

■ 인터넷 통신을 위한 패키지와 클래스

자바에서 인터넷 통신을 하기 위해 사용하는 패키지와 클래스는 다음과 같다.

- **java.net: java.net** 패키지는 자바에서 네트워크 통신 기능을 구현하는 핵심 클래스를 제공한다. 이 패키지는 네트워크 리소스를 나타내는 클래스와 네트워크 소켓을 활용한 통신을 지원하는 클래스를 제공하며, 소켓 프로그래밍을 포함하여 클라이언트/서버 애플리케이션을 구축하는 데 필수적인 패키지이다.
- **java.net.Socket: Socket** 클래스는 네트워크 통신에서 클라이언트 측의 통신 소켓을 나타낸다. 이는 TCP 기반의 연결 지향 connection-oriented 통신을 구현하며, 클라이언트가 서버에 연결을 요청하고, 데이터 스트림을 통해 양방향으로 데이터를 주고받는 데 사용된다. 클라이언트는 **Socket** 객체를 생성한 후 서버의 IP 주소와 포트 번호로 `connect()` 작업을 수행하여 연결을 설정한다. 연결이 성공하면, 이 소켓을 통해 **InputStream**과 **OutputStream**을 얻어 서버와 데이터를 송수신할 수 있다.
- **java.net.ServerSocket: ServerSocket** 클래스는 네트워크 통신에서 서버 측의 통신 엔드포인트 역할을 하며, 클라이언트의 연결 요청을 수신 대기 listen 하고 수락 accept 하는 데 사용한다. 서버는 이 클래스의 객체를 생성하고 특정 포트 번호에 `bind()` 하여 클라이언트의 연결을 기다린다. 가장 중요한 메서드는 `accept()`로, 클라이언트의 연결 요청이 들어올 때까지 실행을 블록

block 하고 기다렸다가, 요청이 들어오면 클라이언트와 실제 데이터 통신을 수행할 새로운 소켓 객체를 반환한다.

- **java.net.InetSocketAddress**: InetSocketAddress는 IP 주소와 포트 번호를 결합한 네트워크 주소 정보를 나타내는 클래스이다. 이는 소켓을 바인딩하거나 연결할 때 필요한 종착지 주소를 명시적으로 정의할 때 사용한다. IP 주소는 InetAddress 클래스를 통해 관리되며, InetSocketAddress는 이 IP 주소와 통신 포트 번호를 포함하는 구조체 역할을 한다. 클라이언트 측에서는 서버의 주소(IP + Port)를 명시하고, 서버 측에서는 일반적으로 포트 번호만 명시하여 모든 IP 인터페이스로부터의 연결을 수락하도록 설정한다.

2. 서버 소켓 통신

[예제 15-1]은 서버가 클라이언트의 연결을 수락하고 데이터를 전송하는 코드이다.

예제 15-1 서버 소켓 코드 분석하기

ServerSocketExample.java

```
01 package sec02.exam01;
02
03 import java.io.OutputStream;
04 import java.net.ServerSocket;
05 import java.net.Socket;
06 import java.net.InetSocketAddress;
07
08 public class ServerSocketExample {
09     public static void main(String[] args) throws Exception {
10         ServerSocket sd = new ServerSocket();
11         InetSocketAddress isa = new InetSocketAddress(11234);
12
13         sd.bind(isa);
14
15         System.out.println("[서버] 시작됨. 클라이언트 연결 대기 중!");
16
17         while(true) {
18             // 서버는 데몬(Daemon)으로 계속 반복 수행
19             Socket sa = sd.accept(); // 클라이언트 연결 수락 및 통신 소켓 반환
20             InetSocketAddress clientAddress = (InetSocketAddress)
21             sa.getRemoteSocketAddress();
22             System.out.println("[서버] 연결 수락: " + clientAddress.
23             getHostString());
24
25             OutputStream os = sa.getOutputStream();
```

```

23         String message = "Hello Client from Server!";    // 전송할 메시지
24         byte[] bytes = message.getBytes("UTF-8");
25
26         os.write(bytes);                                // 데이터 전송
27         os.flush();                                    // 버퍼에 남아있는 데이터 강제 전송
28         System.out.println("[서버] 데이터 전송 완료: [" + message + ""]);
29
30         os.close();
31         sa.close();
32         // 참고: sd.close()는 서버 서비스를 완전히 종료할 때 사용
33     }
34 }
35 }

```

[서버] 시작됨. 클라이언트 연결 대기 중!

10행 `ServerSocket sd = new ServerSocket()`은 클라이언트의 연결 요청을 받아들이기 위한 대기 소켓(listening socket)을 생성한다. 이 소켓은 연결을 수락하는 역할만 수행한다.

11행 `new InetSocketAddress(11234)`에서 `InetSocketAddress`는 일반적으로 IP 주소를 생략하고 포트 번호만 지정한다. IP 주소를 생략하면 서버는 해당 시스템에 할당된 모든 네트워크 인터페이스(모든 IP 주소)로부터 들어오는 연결을 수락한다. 이는 0.0.0.0 주소로 바인딩하는 것과 동일한 효과를 가진다.

13행 `sd.bind(isa)`는 생성된 `ServerSocket`에 포트 번호 '11234'를 할당(바인딩)하고, 이 포트를 통해 연결 요청을 들을 준비를 마친다. 이 과정은 서버가 시작될 때 단 한 번만 수행되며, 15행에서 서버가 준비되었음을 출력한다.

17행의 `while(true)`는 무한 루프이다. 서버는 한 번 실행되면 종료되지 않고, 여러 클라이언트의 요청을 지속적으로 처리한다. 이렇게 종료되지 않고 계속 작동하는 프로세스를 데몬 프로세스라고 부른다. 따라서 서버 코드에서 `sd.close()`는 사용되지 않는다.

18행의 `sd.accept()`가 클라이언트-서버 통신에서 가장 중요한 부분으로, 이 메서드는 클라이언트로부터 연결 요청이 올 때까지 스레드를 멈추고 대기한다. 연결 요청이 들어오면, `accept()`는 클라이언트와 실제로 데이터를 송수신하는 데 사용하는 새로운 통신 소켓 `sa`를 생성하여 반환한다. 이때, `sa`는 `sd`와 다르다는 것에 주의해야 한다. `sd`는 오직 연결을 수락(listen/accept)하는 역할을 하고, `sa`는 클라이언트와 데이터를 일대일로 주고받는 역할을 한다.

22행의 `OutputStream os = sa.getOutputStream()`은 클라이언트에 데이터를 전송하기 위해 연결된 통신 소켓(sa)에서 출력 스트림을 얻는다. 이때 전송할 데이터를 바이트 배열로 변환하여 출력 스트림에 쓰는 것이 26행의 `os.write(bytes)`이다. 27행의 `os.flush()`는 출력 스트림 버퍼에 데이터가 남아있을 경우 이를 강제로 전송한다.

30행의 `os.close()`는 출력 스트림을 닫고, 31행의 `sa.close()`는 클라이언트와의 일대일 통신을 위해 사용된 소켓(sa)을 닫아 해당 클라이언트와의 연결을 종료하고 자원을 해제한다.

[예제 15-1]은 `while(true)` 루프 내에서 `sd.close()`를 호출하지 않는다. 그 이유는 `ServerSocket` 객체인 `sd`가 계속 활성화되어 있어야만 루프의 다음 반복에서 새로운 클라이언트의 연결(`sd.accept()`)을 기다릴 수 있기 때문이다. 따라서 `sd.close()`를 호출하는 것은 서버가 서비스를 완전히 종료하겠다는 의미이며, 더 이상 클라이언트를 받을 수 없게 된다.

Note

listen()

네트워크 통신에서 운영체제는 `listen()` 함수를 실행한다. 그러나 자바는 `ServerSocket` 객체가 생성되는 순간, 내부적으로 `listen()` 함수를 실행하기 때문에 개발자가 별도로 호출할 필요가 없다.

3. 클라이언트 소켓 통신

[예제 15-2]는 클라이언트가 서버에 연결을 요청하고 데이터를 수신하는 [예제 15-1]과 연결하여, 서버가 전송한 메시지를 출력한다.

예제 15-2 클라이언트 코드 분석하기

ClientSocketExample.java

```
01 package sec02.exam02;
02
03 import java.io.InputStream;
04 import java.net.Socket;
05 import java.net.InetSocketAddress;
06
07 public class ClientSocketExample {
08
09     public static void main(String[] args) throws Exception {
10         System.out.println("[클라이언트] 서버 연결 시도 중!");
11
12         InetSocketAddress isa = new InetSocketAddress("127.0.0.1", 11234);
13         Socket sd = new Socket();
14         sd.connect(isa);
15     }
16 }
```

```

16         System.out.println("[클라이언트] 서버 연결 성공!");
17
18         InputStream is = sd.getInputStream();
19         byte[] buffer = new byte[1024];
20         int readByte = is.read(buffer);
21
22         if (readByte != -1) {
23             String message = new String(buffer, 0, readByte, "UTF-8");
24             System.out.println("[클라이언트] 수신 메시지: " + message);
25         }
26
27         is.close();
28         sd.close();
29
30         System.out.println("[클라이언트] 연결 종료.");
31     }
32 }

```

클라이언트 콘솔 화면

```

[클라이언트] 서버 연결 시도 중!
[클라이언트] 서버 연결 성공!
[클라이언트] 수신 메시지: Hello Client from Server!
[클라이언트] 연결 종료.

```

서버 콘솔 화면

```

[서버] 시작됨. 클라이언트 연결 대기 중!
[서버] 연결 수락: 127.0.0.1
[서버] 데이터 전송 완료: [Hello Client from Server!]

```

12행 `InetSocketAddress isa = new InetSocketAddress("127.0.0.1", 11234)`는 서버 주소를 설정한다. 여기서 `127.0.0.1`은 루프백(loop-back) 주소라고 불리는 특별한 IP 주소이며, 이 주소는 항상 현재 컴퓨터 자신을 가리킨다. 실제 서비스 환경에서는 서버 코드가 실행 중인 컴퓨터의 실제 IP 주소가 들어가야 한다. 그러나 루프백 주소를 사용하면 클라이언트와 서버 프로그램을 같은 컴퓨터에서 실행하면서도 네트워크 통신을 하는 것처럼 시뮬레이션할 수 있다. 이때, `127.0.0.1`과 같은 루프백 주소 대신에 `"localhost"`를 사용해도 같은 결과를 얻을 수 있다. 또한 `11234`는 서버 애플리케이션이 클라이언트의 연결을 기다리고 있는 포트 번호이며, 클라이언트와 서버는 이 포트 번호를 통해 서로를 찾아 통신한다.

14행 `sd.connect(isa)`는 클라이언트 소켓이 `isa`에 설정된 주소를 가진 서버에 연결을 시도한다. 연결이 성공하면 클라이언트와 서버 사이에 가상의 통신선이 설정된다.

18행 `InputStream is = sd.getInputStream()`은 연결된 소켓을 통해 서버로부터 데이터를 수신하기 위한 입력 스트림 객체를 만든다. 20행 `is.read(buffer)`는 입력 스트림을 통해 서버가 보낸 데이터를 읽어 `byte` 배열에 저장한다.

27행 `is.close()`는 입력 스트림을 닫고, 28행 `sd.close()`는 사용을 마친 소켓을 닫아 연결을 종료한다.

4. 클라이언트-서버 통신의 실행과 종료 방법

유닉스(또는 윈도우의 터미널) 상에서 서버와 클라이언트 소켓 코드를 실행하는 절차를 살펴보자. 가장 먼저 서버와 클라이언트의 자바 소스 코드를 JVM이 실행 가능한 바이트코드(.class 파일)로 컴파일한다.

```
$> javac ServerSocketTest.java
$> javac ClientSocket.java
```

서버(데몬)는 클라이언트의 연결 요청을 항상 기다려야 하므로, 클라이언트보다 먼저 실행되어야 한다. 또한, 서버는 터미널을 점유하지 않고 계속 실행되어야 하므로 다음과 같이 `&`를 사용하여 백그라운드 Background로 실행해야 한다.

```
$> java ServerSocketTest &
```

`&`기호는 유닉스 계열 셸에서 프로세스를 백그라운드로 실행하도록 지시하기 때문에 사용자는 터미널에서 즉시 다른 명령을 입력할 수 있게 된다. 서버는 이제 포트 11234에서 클라이언트의 `connect()` 호출을 기다리는 상태가 된다. 이후 클라이언트 프로그램을 실행하면 서버에 접속하여 데이터를 받고 즉시 종료된다.

```
$> java ClientSocket
[클라이언트] 서버 연결 시도 중!
[클라이언트] 서버 연결 성공!
[클라이언트] 수신 메시지: Hello Client from Server!
[클라이언트] 연결 종료.
```


클라이언트가 종료된 후에도 서버 프로세스는 무한 루프를 돌며 여전히 활성화되어 있기 때문에, 클라이언트 프로그램을 여러 번 다시 실행해도 서버는 각 요청에 응답하며 동일한 결과를 반복적으로 출력한다. 따라서, 서버 코드는 백그라운드에서 계속 실행되는 데몬 프로세스이므로, 명시적으로 중지 명령을 내려야 한다. `ps -ef` 명령어는 시스템에서 실행 중인 모든 프로세스의 목록을 보여준다. 이 목록을 `grep` 명령어를 사용하여 실행 중인 서버 코드 이름으로 필터링하여 프로세스 ID(PID)를 확인한다.

```
$> ps -ef | grep ServerSocketTest
```

확인된 PID를 사용하여 `kill` 명령어로 해당 프로세스에 종료 시그널을 보낸다. 만약 PID가 '12345'일 경우 다음과 같이 명령을 내린다.

```
$> kill 12345
```

이클립스로 실행한 프로그램은 이클립스 콘솔 창에서 종료시킬 수 있다. 이클립스 하단의 콘솔 탭을 활성화한 후, 오른쪽 상단에 있는 빨간색 사각형 아이콘()을 클릭한다. 이 아이콘은 현재 콘솔에 연결된 실행 중인 자바 프로세스를 강제 종료하는 역할을 한다.

2 URL을 이용한 통신

자바의 `URL` 및 `URLConnection` 클래스를 사용하여 웹 서버에 접속하고 데이터를 읽어오는 기본적인 클라이언트 통신 방법을 살펴보자. 이 방식은 앞서 살펴본 소켓 프로그래밍보다 훨씬 간편하게 웹 리소스에 접근할 수 있다. `URL` Uniform Resource Locator은 인터넷상의 모든 자원을 나타내는 표준화된 주소 체계이다. 웹페이지, 이미지, 파일 등 인터넷에 존재하는 모든 콘텐츠는 고유한 URL을 가지고 있으며, 다음과 같은 주요 구성 요소로 이루어져 있다.

- **프로토콜**: 자원에 접근하는 데 사용하는 규칙이나 방법을 의미한다. 예) `http`, `https`, `ftp` 등
- **호스트명**: 자원이 위치한 서버의 이름 또는 IP 주소를 나타낸다. 예) `www.google.com`, `www.naver.com` 등
- **포트 번호**: 서버 내 특정 애플리케이션을 식별하는 번호이며, `http`의 기본 포트는 80번이므로 보통 생략한다.

자바의 `java.net.URL` 클래스는 이러한 URL 주소를 객체화하여, 해당 주소의 자원에 쉽게 접근하고 정보를 추출할 수 있는 기능을 제공한다. [예제 15-3]은 구글(`http://www.google.com`) 주소에 접속하여 서버로부터 받은 HTML 데이터를 읽어와 콘솔에 출력하는 코드이다.

예제 15-3 URL을 이용하여 통신하기

UrlClientExample.java

```
01 package sec02.exam03;
02
03 import java.io.BufferedReader;
04 import java.io.InputStreamReader;
05 import java.net.URL;
06 import java.net.URLConnection;
07
08 public class UrlClientExample {
09     public static void main(String[] args) {
10         try {
11             URL url = new URL("http://www.google.com");
12
13             URLConnection connection = url.openConnection();
14
15             BufferedReader in = new BufferedReader(new InputStreamReader
16 (connection.getInputStream()));
17
18             String inputLine;
19             System.out.println("--- Google.com에서 받은 응답 ---");
20             while ((inputLine = in.readLine()) != null) {
21                 System.out.println(inputLine);
22             }
23             in.close();
24         } catch (Exception e) {
25             e.printStackTrace();
26         }
27     }
28 }
```

```
--- Google.com에서 받은 응답 ---
<!doctype html><html itemscope="" itemtype="http://schema.org/WebPage" lang="ko"><head><meta
content="text/html; charset=UTF-8" http-equiv="Content-Type"><meta content="/images/branding/
google/1x/google_standard_color_128dp.png" itemprop="image"><title>Google</title>.... 이하 생략
```

11행 `URL url = new URL("http://www.google.com")`은 접근하고자 하는 웹 자원의 주소, 즉 `http://www.google.com`을 사용하여 URL 객체를 생성한다. 이 객체는 해당 주소에 대한 모든 정보를 포함하며, 주소가 잘못된 형식일 경우 `MalformedURLException`이 발생할 수 있다.

13행 `URLConnection connection = url.openConnection()`은 URL 객체의 `openConnection()` 메서드를 호출하여 해당 자원과의 통신을 처리할 `URLConnection` 객체를 얻는다. 이 객체는 서버와 실제 통신 채널을 설정하고 데이터를 주고받는 데 필요한 모든 세부 작업을 담당하는 추상 클래스이다. 이 시점에서는 아직 실제 네트워크 연결(소켓 연결)이 이루어지지 않았으며, 연결을 위한 준비만 완료된 상태이다.

15행 `BufferedReader in = new BufferedReader(new InputStreamReader(connection.getInputStream()))`은 서버로부터 전송된 데이터를 효율적으로 읽어오기 위해 여러 개의 스트림 클래스를 조합하여 사용한다.

- `connection.getInputStream()`: 서버와 연결이 수립되고, 서버가 보낸 응답 데이터를 읽어오는 바이트 기반의 `InputStream` 객체를 얻는다.
- `new InputStreamReader()`: 바이트 기반의 `InputStream`을 문자 기반의 `Reader`로 변환한다. 웹 응답은 대부분 텍스트(HTML)이므로, 바이트를 문자로 변환하여 처리하는 것이 편리하다. 이 과정에서 문자 인코딩(예 UTF-8)을 지정할 수 있으나, 여기서는 시스템의 기본 인코딩을 사용한다.
- `new BufferedReader()`: `InputStreamReader`를 `BufferedReader`로 감싸서 버퍼링 기능을 추가한다. `BufferedReader`는 데이터를 한 줄씩(`readLine()`) 효율적으로 읽을 수 있는 편리한 기능을 제공하여 입출력 성능을 향상시킨다.

19행 `while ((inputLine = in.readLine()) != null)`은 `BufferedReader`의 `readLine()` 메서드를 사용하여 서버로부터 받은 응답 데이터를 한 줄씩 읽어 `inputLine` 변수에 저장한다. `readLine()` 메서드는 더 이상 읽을 데이터가 없으면 `null`을 반환하므로, 루프는 서버가 응답을 모두 보낼 때까지 계속 실행된다. 20행에서 루프 내부에서는 읽어 들인 `inputLine`을 콘솔에 출력한다. 이 출력 내용은 `www.google.com`의 HTML 소스 코드이다.

22행 `in.close()`는 데이터 읽기를 완료한 후, 사용했던 `BufferedReader`를 닫는다. `BufferedReader`를 닫으면 그 안에 감싸져 있던 하위 스트림들도 자동으로 닫히며, 소켓 연결을 종료하고 관련된 시스템 자원을 운영체제에 반납한다.

3 UDP를 이용한 통신

1. UDP의 이해

UDP (User Datagram Protocol)를 사용하는 서버 프로그램의 구현과 작동 원리를 살펴보자. UDP는 TCP와 함께 인터넷 프로토콜 위에서 작동하는 전송 계층 프로토콜이지만, TCP(소켓 통신)와는 근본적으로 다른 통신 방식이다. UDP의 가장 큰 특징으로는 비연결형 (Connectionless) 통신 방식을 사용한다는 것이다.

UDP는 클라이언트와 서버 사이에 미리 연결을 설정할 필요 없이, 데이터를 데이터그램 (datagram)이라는 독립적인 패킷 단위로 목적지 주소와 포트 번호를 붙여 발송한다. 편지를 보낼 때처럼 보내는 쪽은 잘 전달되었는지 확인하지 않고, 받는 쪽은 해당 패킷이 도착하면 특별한 작업 없이 데이터를 처리한다. [표 15-1]은 TCP와 UDP의 차이를 비교한 것이다.

표 15-1 TCP와 UDP 비교

특징	TCP (Transmission Control Protocol)	UDP (User Datagram Protocol)
연결 방식	연결 지향	비연결형
신뢰성	높음 (데이터 순서 보장, 오류 제어, 재전송)	낮음 (데이터 손실 및 순서 보장하지 않음)
속도	상대적으로 느림 (오버헤드 발생)	상대적으로 빠름 (간단한 헤더)
사용 용도	웹 브라우징 (HTTP), 파일 전송, 이메일	실시간 스트리밍, 온라인 게임, DNS

UDP는 속도가 중요하고 데이터 손실이 허용되는 애플리케이션에 주로 사용되며, Datagram Socket과 DatagramPacket 클래스를 사용하여 구현한다.

2. UDP를 이용한 서버 통신

[예제 15-4]는 포트 '11234'에서 클라이언트의 UDP 데이터그램을 수신하고, 받은 메시지를 클라이언트로 되돌려 주는 코드이다.

예제 15-4 UDP를 이용하여 서버 통신하기

UdpServerExample.java

```
01 package sec02.exam04;
02
03 import java.net.DatagramPacket;
04 import java.net.DatagramSocket;
05 import java.net.InetAddress;
06
07 public class UdpServerExample {
08     public static void main(String[] args) {
09         try {
```

```

10         DatagramSocket socket = new DatagramSocket(11234);
11         System.out.println("[UDP 서버] 시작되었습니다. 포트: 11234");
12
13         byte[] receiveData = new byte[1024];
14
15         while(true) {
16             DatagramPacket receivePacket = new DatagramPacket(receiveData,
17 receiveData.length);
18
19             socket.receive(receivePacket);
20
21             String message = new String(receivePacket.getData(), 0,
22 receivePacket.getLength());
23             InetAddress clientAddress = receivePacket.getAddress();
24             int clientPort = receivePacket.getPort();
25
26             System.out.println("[수신] " + clientAddress + ":" + clientPort
27 + " : " + message);
28
29             String response = "Echo: " + message;
30             byte[] sendData = response.getBytes();
31             DatagramPacket sendPacket = new DatagramPacket(sendData,
32 sendData.length, clientAddress, clientPort);
33
34             socket.send(sendPacket);
35         }
36     } catch (Exception e) {
37         e.printStackTrace();
38     }
39 }

```

```

-
[UDP 서버] 시작되었습니다. 포트: 11234

```

10행의 `DatagramSocket`은 UDP 통신을 담당하는 소켓 객체이며, TCP의 `ServerSocket` 및 `Socket`의 역할을 모두 수행한다. 따라서 `DatagramSocket socket = new DatagramSocket(11234)`는 객체를 생성하는 동시에 포트 번호 '11234'로 바인딩 된다. TCP와 달리 UDP는 연결 수립 과정이 없기 때문에 이 `DatagramSocket` 하나로 계속해서 데이터를 수신하고, 응답할 수 있다.

16행 `DatagramPacket receivePacket = new DatagramPacket(receiveData, receiveData.length)`에서 `DatagramPacket`은 UDP 통신에서 데이터를 싣는 패킷 단위 객체이다. TCP가 스트림 기반인 반면, UDP는 패킷 기반이다. 데이터를 수신할 때는 수신할 바이트 배열(`receiveData`)과 그 길이를 미리 지정하여 `DatagramPacket` 객체를 생성한다. 이 패킷은 데이터를 받아 담는 컨테이너 역할을 한다.

18행 `socket.receive(receivePacket)`은 클라이언트로부터 데이터그램이 도착할 때까지 블록 상태로 대기한다. 데이터가 도착하면, 패킷의 내용은 `receivePacket`의 내부 바이트 배열에 채워지며, 동시에 패킷을 보낸 클라이언트의 IP 주소와 포트 번호 정보도 이 `receivePacket` 객체 내부에 저장된다. UDP는 비연결형이므로, 응답을 보내기 위해서는 수신된 패킷에서 추출한 클라이언트의 주소와 포트 정보를 반드시 사용해야 한다.

26행은 수신된 메시지에 "Echo: "를 붙여 응답 문자열을 만들고 바이트 배열(`sendData`)로 변환한다. 이후 28행에서 데이터를 담을 `DatagramPacket` 객체를 생성할 때, 전송할 데이터, 데이터의 길이 외에도 데이터를 보낼 최종 목적지인 `clientAddress`(클라이언트 IP)와 `clientPort`(클라이언트 포트)를 명시적으로 지정한다.

30행 `socket.send(sendPacket)`은 생성된 송신 패킷을 네트워크로 전송한다. 이로써 서버는 클라이언트로부터 받은 메시지를 다시 클라이언트로 돌려보내는 역할을 수행한다.

[예제 15-4]의 서버 역시 `while(true)` 루프 안에서 작동하므로, 강제 종료 명령이 내려지기 전까지는 계속해서 클라이언트의 UDP 데이터그램을 수신하고 응답하는 서비스를 지속한다.

3. UDP를 이용한 클라이언트 통신

UDP 클라이언트는 서버로 데이터그램 패킷을 전송하고, 서버로부터 응답을 받는다. [예제 15-5]는 사용자로부터 메시지를 입력받아 UDP 서버(포트 11234)로 전송하고, 서버의 응답을 수신하여 출력하는 코드이다.

예제 15-5 UDP를 이용하여 클라이언트 통신하기

UdpClientExample.java

```
01 package sec02.exam05;
02
03 import java.net.DatagramPacket;
04 import java.net.DatagramSocket;
05 import java.net.InetAddress;
06 import java.util.Scanner;
07
```

```

08 public class UdpClientExample {
09     public static void main(String[] args) {
10         try {
11             DatagramSocket socket = new DatagramSocket();
12             InetAddress serverAddress = InetAddress.getByName("127.0.0.1");
13             int serverPort = 11234;
14
15             Scanner scanner = new Scanner(System.in);
16
17             while(true) {
18                 System.out.print("메시지 입력: ");
19                 String message = scanner.nextLine();
20
21                 if (message.equalsIgnoreCase("exit")) {
22                     break;
23                 }
24
25                 byte[] sendData = message.getBytes();
26                 DatagramPacket sendPacket = new DatagramPacket(sendData,
27                     sendData.length, serverAddress, serverPort);
28
29                 socket.send(sendPacket);
30
31                 byte[] receiveData = new byte[1024];
32                 DatagramPacket receivePacket = new DatagramPacket(receiveData,
33                     receiveData.length);
34                 socket.receive(receivePacket);
35
36                 String serverResponse = new String(receivePacket.getData(), 0,
37                     receivePacket.getLength());
38                 System.out.println("[서버 응답]: " + serverResponse);
39             }
40             socket.close();
41             scanner.close();
42         } catch (Exception e) {
43             e.printStackTrace();
44         }
45     }
46 }

```

```

메시지 입력: Miracle Java
[서버 응답]: Echo: Miracle Java
메시지 입력: exit

```

11행 `DatagramSocket socket = new DatagramSocket()`은 UDP 통신을 위한 `DatagramSocket` 객체를 생성하며, 클라이언트 측에서는 포트 번호를 지정하지 않는다. 포트 번호를 지정하지 않으면 운영체제가 자동으로 사용 가능한 임시 포트를 할당하며, 이 임시 포트는 서버가 클라이언트로 응답을 보낼 때 사용된다.

12행 `InetAddress serverAddress = InetAddress.getByName("127.0.0.1")`은 메시지를 보낼 서버의 IP 주소를 나타내는 `InetAddress` 객체를 생성한다. 여기서는 서버와 클라이언트가 같은 컴퓨터에 있다는 의미의 루프백 주소(127.0.0.1)를 사용한다.

17행 `while(true)`은 클라이언트는 사용자가 'exit'을 입력하기 전까지 계속해서 메시지를 보내고 서버 응답을 받을 수 있는 무한 루프이다.

26행은 사용자로부터 받은 문자열 메시지를 바이트 배열(`sendData`)로 변환한다. 네트워크를 통해 전송되는 데이터는 항상 바이트 형태로 처리되어야 한다. `DatagramPacket sendPacket = new DatagramPacket()`은 전송할 데이터, 데이터의 길이, 데이터의 목적지 주소(`server Address`)와 포트 번호(`serverPort`)를 명시하여 송신용 데이터그램 패킷을 생성한다. UDP는 비연결형이므로, 데이터를 보낼 때마다 목적지 정보를 패킷에 포함시켜야 한다.

28행 `socket.send(sendPacket)`은 생성된 패킷을 지정된 서버 주소로 네트워크를 통해 발송한다.

30행과 31행은 서버 응답을 받을 빈 바이트 배열과 이를 받을 수신용 `DatagramPacket`을 생성한다. 32행 `socket.receive(receivePacket)`은 서버의 응답 패킷이 도착할 때까지 클라이언트 프로그램은 블록 상태로 대기한다. 응답이 도착하면 패킷의 내용이 `receivePacket`에 채워지고, 수신된 데이터는 다시 문자열로 변환되어 클라이언트 콘솔에 출력된다.

37행과 38행은 루프를 벗어나는 경우에 `DatagramSocket`과 `Scanner` 객체를 닫아 사용된 네트워크 포트 및 시스템 자원을 운영체제에 반납한다. 이는 모든 소켓 통신에서 중요한 마무리 절차이다.

생성형 AI와 공부하기

자바를 이용한 클라이언트-서버 통신 코드의 특징을 설명하고, TCP 대신 UDP를 사용할 때의 차이점을 설명해줘.

요약

- 01 컴퓨터 네트워크란 두 대 이상의 컴퓨터가 데이터를 주고받는 통신 시스템을 의미하며, 통신을 위한 약속을 프로토콜이라 부른다. 미국 국방성은 서로 호환되지 않는 LAN을 묶어 하나의 네트워크로 만들기 위한 연구를 진행하였는데, 이 네트워크가 전 세계에 공개되어 지금의 인터넷이 된다. 인터넷을 위해 만들어진 IP는 네트워크와 네트워크를 넘어 목적지까지 데이터를 전송하는 프로토콜이며, TCP는 보내진 데이터가 원래의 상태로 도착하는지를 감시하는 역할을 하는 프로토콜이다.
- 02 종료되지 않은 채 서비스를 계속 진행하는 프로그램을 데몬이라 부른다. 클라이언트-서버 구조에서 클라이언트란 서비스를 요청하는 컴퓨터이며, 서버는 데몬이 설치된 컴퓨터이다. 인터넷을 사용하는 각 컴퓨터를 구분할 때 사용하는 주소가 IP 주소이며, TCP가 응용 프로그램을 구분하기 위해 사용하는 주소 체계가 포트이다. 포트에 멀티탭을 연결하듯 각 사용자에게 개별적인 연결 공간을 나눠주는데, 이를 소켓이라 부른다.
- 03 자바 소켓 프로그래밍에서 `sd = Socket()`을 호출하면 소켓 기술자를 반환하며, 이후 모든 작업은 소켓 기술자를 통해 이루어진다. 통신을 마친 후 `close()`를 사용하여 소켓 기술자를 닫아 연결을 종료한다. `sd = Socket()` 이후에 클라이언트가 서버에 연결을 요청하는 `connect()`와 클라이언트의 요청을 수락하는 서버의 `accept()`가 추가된다.
- 04 자바는 네트워크 데이터를 읽고 쓸 때 스트림 기반 입출력을 사용하며, `java.net` 패키지의 `java.net.Socket`, `java.net.ServerSocket`, `java.net.InetSocketAddress` 등의 클래스를 사용한다. 네트워크 통신을 위해선 서버 프로세스가 클라이언트보다 먼저 실행되어 클라이언트의 연결을 기다려야 한다.
- 05 URL은 인터넷상의 모든 자원을 나타내는 표준화된 주소 체계이며, 자바의 `java.net.URL` 클래스는 이러한 URL 주소를 객체화하여, 해당 주소의 자원에 쉽게 접근하고 정보를 추출할 수 있는 기능을 제공한다.
- 06 UDP는 비연결형 통신 방식이기 때문에 클라이언트와 서버 사이에 미리 연결을 설정할 필요 없이, 독립적인 패킷 단위로 목적지 주소와 포트 번호를 붙여 발송한다. UDP는 속도가 중요하고 데이터 손실이 허용되는 애플리케이션에 주로 사용되며, `DatagramSocket`과 `DatagramPacket` 클래스를 사용하여 구현한다.

연습 문제

01 통신을 위한 약속을 의미하는 단어를 찾으시오.

- ① 프로토콜 ② 데몬 ③ TCP ④ IP

02 종료되지 않은 채 서비스를 계속 진행하는 프로그램을 의미하는 단어를 찾으시오.

- ① 프로토콜 ② 데몬 ③ TCP ④ IP

03 네트워크와 네트워크를 넘어 목적지까지 데이터를 전송하는 프로토콜을 찾으시오.

- ① UDP ② URL ③ TCP ④ IP

04 보낸 데이터가 원래의 상태로 도착하는지를 감시하는 역할을 하는 프로토콜을 찾으시오.

- ① UDP ② URL ③ TCP ④ IP

05 IP에서 올라온 패킷을 특별한 작업 없이 응용 프로그램에 전달하는 프로토콜을 찾으시오.

- ① UDP ② URL ③ TCP ④ LAN

06 TCP가 응용 프로그램을 구분하기 위해 사용하는 주소 체계를 의미하는 단어를 찾으시오.

- ① 프로토콜 ② 데몬 ③ 소켓 ④ 포트

07 같은 포트에 연결되어 여러 통신 작업을 동시에 처리할 수 있는 소프트웨어적인 접속 장치를 의미하는 단어를 찾으시오.

- ① 프로토콜 ② 데몬 ③ 소켓 ④ 서버

08 자바 클라이언트 소켓 코드 중 소켓 기술자를 얻기 위해 사용하는 메서드를 찾으시오.

- ① socket() ② close() ③ connect() ④ read()

09 자바 클라이언트 소켓 코드 중 서버에 연결을 요청하는 메서드를 찾으시오.

- ① `socket()` ② `close()` ③ `connect()` ④ `read()`

10 자바 클라이언트 소켓 코드 중 소켓 기술자를 닫아 연결을 종료하는 메서드를 찾으시오.

- ① `socket()` ② `close()` ③ `connect()` ④ `read()`

11 자바 서버 소켓 코드 중 소켓 기술자를 얻기 위해 사용하는 메서드를 찾으시오.

- ① `listen()` ② `accept()`
③ `bind()` ④ `serversocket()`

12 자바 서버 소켓 코드 중 생성된 소켓에 IP 주소와 포트 번호를 할당하는 메서드를 찾으시오.

- ① `serversocket()` ② `accept()`
③ `bind()` ④ `listen()`

13 자바 서버 소켓 코드 중 클라이언트의 연결 요청이 들어오면 이를 수락하는 메서드를 찾으시오.

- ① `serversocket()` ② `accept()`
③ `bind()` ④ `listen()`

14 자바 클라이언트의 `connect()` 메서드에 대응하는 서버 메서드를 찾으시오.

- ① `serversocket()` ② `accept()`
③ `bind()` ④ `listen()`

15 인터넷상의 모든 자원을 나타내는 표준화된 주소 체계를 의미하는 단어를 찾으시오.

- ① UDP ② URL ③ TCP ④ IP

데이터베이스와 JDBC

01 데이터베이스

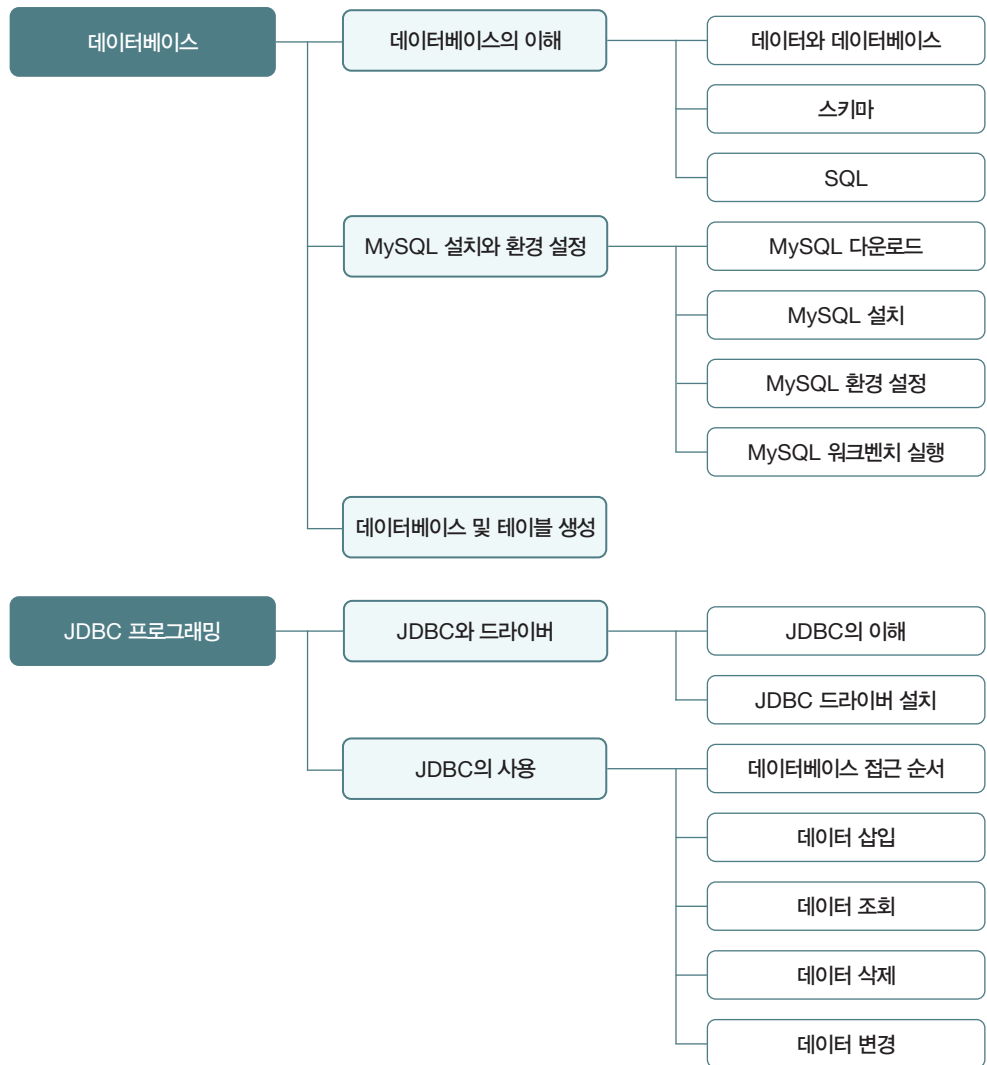
02 JDBC 프로그래밍

요약

연습문제

학습 목표

- DBMS, 스키마, SQL과 같이 데이터베이스와 관련된 다양한 용어를 이해한다.
- MySQL 설치 방법을 배우고 테이블을 생성한 후 데이터를 삽입한다.
- JDBC를 사용하여 데이터의 삽입, 조회, 삭제, 변경 방법을 학습한다.



1 데이터베이스의 이해

1. 데이터와 데이터베이스

슈퍼마켓에서 물건을 구매하는 장면을 떠올려보자. 선반에는 다양한 음료수가 가득 진열되어 있고, 그 앞에 가격표가 붙어 있다. 계산대에는 바코드가 찍힌 영수증이 출력되고, 창고에는 재고가 차곡차곡 쌓여 있다. 이 모든 것이 데이터이다. 상품명, 용량, 가격, 하루 판매량, 남은 재고, 한 병당 이익 같은 정보들이 눈에 보이지 않는 숫자와 문자로 존재한다. 하지만 이러한 데이터는 그 자체로는 큰 의미가 없다. 이 데이터를 잘 모아 정리하고 가공해야만 비로소 매장 운영에 도움이 되는 유용한 정보가 된다. 예를 들어 ‘일일 판매 데이터 그래프’는 판매 기록을 수집하고 계산한 뒤 시각화했기 때문에 점주가 어떤 상품을 추가로 주문해야 할지, 어떤 재고를 보충해야 할지를 판단하는 데 중요한 기준이 된다.

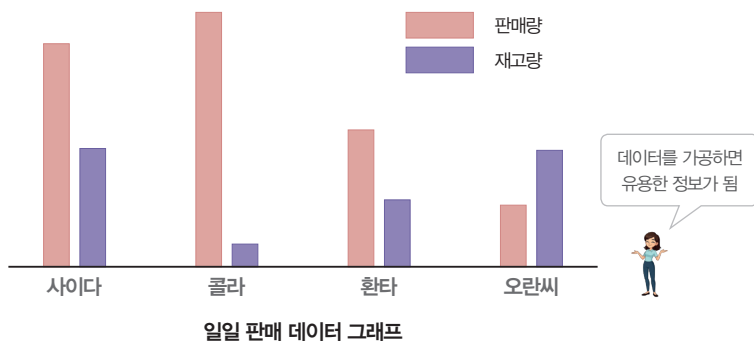


그림 16-1 데이터와 정보

데이터를 잘 관리하려면 체계적인 틀이 필요하다. 과거에는 장부에 주문과 판매 내역을 손으로 적고 계산하는 방식으로 운영할 수 있었지만, 판매 규모가 커지고, 손님들이 한 번에 물려오는 상황이 늘어난다면 수기로 작성하는 방식에는 한계가 있다. 그래서 등장한 것이 바로 데이터베이스이다. 데이터베이스는 ‘데이터를 안전하고 효율적으로 저장하고, 관리할 수 있는 창고’이다. 단순히 데이터를 파일 하나에 저장해 두는 것과 큰 차이가 있는데, 데이터베이스는 데이터의 중복을 줄이고, 데이터의 정확성을 유지하며, 여러 사용자가 동시에 접근해도 충돌이 나지 않도록

돕는다. 로그인 기능을 제공하는 웹사이트에서 수백만 명의 아이디와 비밀번호를 안정적으로 관리할 수 있는 것도 모두 데이터베이스 덕분이다.

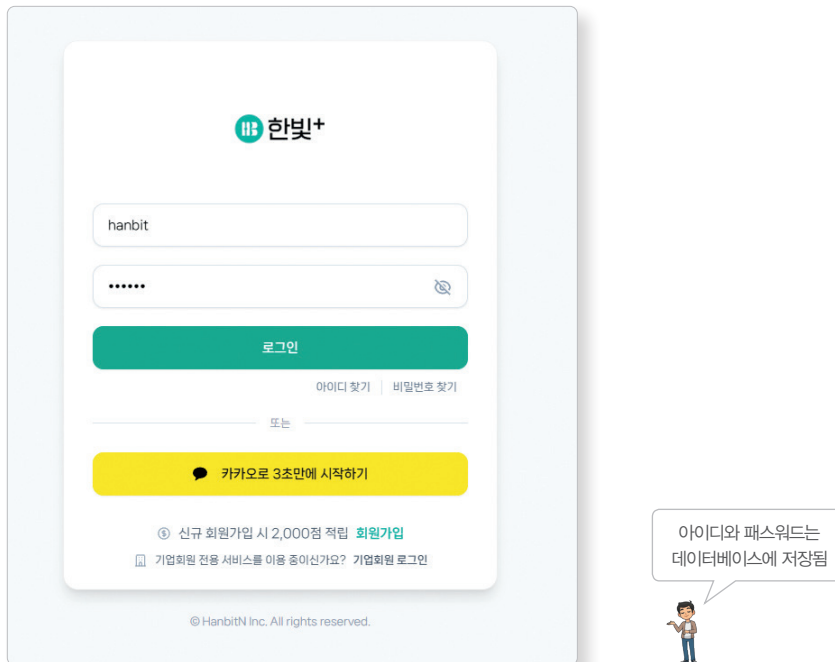


그림 16-2 로그인 페이지와 데이터베이스

데이터베이스 관리 시스템 DataBase Management System, DBMS은 **사용자와 데이터베이스를 연결하는 소프트웨어 시스템이다**. DBMS는 사용자가 데이터의 실제 저장 방식에 대해 직접 알지 않아도, 필요한 작업을 요청할 수 있도록 돕는 중간 관리자 역할을 한다. 운영체제가 하드웨어 자원을 보호하고 응용 프로그램에 적절히 배분하듯, DBMS는 데이터베이스를 보호하고 애플리케이션이 데이터를 사용할 수 있도록 접근 기능을 제공한다. 또한 권한 관리, 보안, 백업과 복구, 동시성 제어와 같은 중요한 기능도 담당한다.

2. 스키마

대표적인 DBMS 소프트웨어에는 Oracle, MySQL, PostgreSQL, MS SQL Server 등이 있다. 이러한 현대의 DBMS는 관계형 모델 relational model을 사용하며, 관계 데이터베이스 관리 시스템 Relation DataBase Management System, RDBMS라 부른다. 관계 모델에서 데이터는 테이블 table 형태로 구성되며, 구성된 테이블들은 서로 간의 키 key를 중심으로 여러 관계를 형성한다. 테이블은 여러 개의 열(필드)과 행(레코드)으로 이루어져 있으며, 각 열은 데이터의 속성 attribute을 의미하고, 각 행은 한 건의 완성된 데이터 집합이다.

예를 들어 ‘사원’ 테이블이 있다면 이름, 부서, 연봉과 같은 열이 있고, 각 사원 한 명 한 명의 정보가 행으로 저장된다. 이때 중요한 것은 스키마(schema)라는 설계도이다. 데이터베이스에서 데이터는 이미 만들어진 틀에 저장되는데, 저장할 데이터의 구조, 데이터와 데이터 간의 관계, 제약조건을 정의한 것이 바로 스키마이다.

스키마는 어떤 테이블이 있고, 각 테이블이 어떤 열을 가지며, 열의 타입과 제약조건은 무엇인지를 정의한다. 스키마는 데이터베이스의 설계도와 같아서, 처음에 스키마를 제대로 만들지 않으면 이후 데이터베이스를 사용할 때 문제가 발생할 수 있다. 예를 들어, 스키마를 잘못 설계하면 저장 공간을 낭비하거나, 혹은 긴 상품명을 저장하지 못하는 등 현실과 맞지 않는 문제가 발생할 수 있다. 따라서 데이터베이스 설계자는 실제로 다룰 데이터의 특성을 관찰한 뒤 적절한 타입과 길이, 제약조건을 설정해야 한다.

[그림 16-3]은 한 회사의 사원 데이터베이스 예를 보여준다. 스키마에서 하나의 데이터를 저장할 수 있는 영역을 열이라 부르는데, 스키마는 여러 개의 열에 대한 정의의 집합이며, 각 열의 타입과 제약조건을 명시한다. 예를 들어, **employees** 테이블의 스키마는 사원 번호를 나타내는 **id** 열, 이름을 나타내는 **name** 열, 부서를 나타내는 **department** 열, 연봉을 나타내는 **salary** 열로 구성된다. 스키마 구조에 맞추어 저장되는 각 영역을 필드(field)라 부르며, 저장된 데이터 묶음 1개를 레코드(record) 또는 행이라고 한다.

열	id	name	department	salary
필드	1	김철수	개발	5000
레코드	2	이영희	인사	4500
레코드	3	박민준	영업	4800

데이터를 저장하는 영역이 필드

그림 16-3 employees 데이터베이스의 예제

스키마를 설계한 뒤 구조에 맞게 데이터베이스에 데이터를 집어넣는다. [그림 16-3]의 **employees** 테이블의 스키마는 다음과 같은 구조로 정의된다.

```
CREATE TABLE employees (
    id INT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(100) NOT NULL,
    department VARCHAR(50),
    salary INT
);
```


이 스키마에서 **id**는 정수(**INT**)여야 하며, **name**은 최대 100글자로 구성되고 빈 값(**NULL**)이 들어올 수 없다. 또한, **department**는 최대 50자이며, **salary**는 정수(**INT**)이다. 관계형 데이터베이스에서 레코드의 데이터를 읽어 오거나 변경할 때는 키를 사용하는데, **employees** 테이블의 경우 키 값은 **id**이다.

3. SQL

컴퓨터에 작업 명령을 내릴 때 프로그래밍 언어를 사용하듯, DBMS에서도 정해진 언어를 사용하여 데이터를 처리한다. 데이터베이스에 저장된 데이터를 처리하는 언어가 바로 **SQL** Structured Query Language이다. SQL은 관계형 데이터베이스와 통신하기 위한 표준 언어이며, SQL을 사용하여 형식에 맞는 질문_{query}을 하면 DBMS가 응답한다.

SQL을 사용하면 테이블을 만들고, 데이터를 삽입하거나 수정하고 조회하며, 권한을 부여하거나 회수하고, 트랜잭션을 제어하는 모든 작업을 수행할 수 있다. 자바 애플리케이션에서는 JDBC를 활용하여 SQL 문을 DBMS에 문자열로 전달하고 그 결과를 받아온다. 이처럼 JDBC는 자바와 DBMS를 연결하는 다리 역할을 한다. 예를 들어 SQL로 ‘김철수’, ‘개발’, ‘5000’ 같은 정보를 JDBC를 통해 DBMS에 전송하면 DBMS가 실제 데이터를 조작한다.

Note

트랜잭션과 ACID

데이터를 안전하게 다루려면 트랜잭션의 개념을 이해하는 것이 필수이다. 트랜잭션은 데이터베이스 상태를 바꾸는 일련의 작업을 하나의 논리적 단위로 묶은 것이다. 예를 들어 한 고객이 천만 원을 다른 계좌로 이체하는 상황에서는 ‘내 계좌에서 천만 원 감소’와 ‘다른 계좌에 천만 원 증가’라는 두 작업이 동시에 이루어져야 한다. 이 두 작업 중 하나만 성공하고 다른 하나가 실패하면 데이터의 일관성이 깨진 상태가 된다. 데이터의 일관성을 유지하려면 트랜잭션은 ACID라 불리는 네 가지 성질을 보장해야 한다. 원자성_{Atomicity}은 트랜잭션 내 모든 작업이 전부 성공하거나 전부 실패해야 함을 의미하고, 일관성_{Consistency}은 트랜잭션 전후에 데이터가 정의된 제약조건을 만족해야 함을 뜻한다. 격리성_{Isolation}은 동시에 실행되는 다른 트랜잭션에 방해받지 않도록 보장해야 함을 의미하고, 영속성_{Durability}은 한 번 커밋_{commit}된 결과는 시스템 오류가 나더라도 보존되어야 한다는 의미이다.

2 MySQL 설치와 환경 설정

1. MySQL 다운로드

MySQL은 Oracle에서 관리하는 RDBMS로 가장 인기 있는 오픈 소스 중 하나이다. MySQL Installer를 사용하면 서버와 GUI 관리 도구인 MySQL 워크벤치를 한 번에 설치할 수 있어 편리

하다. JDBC 실습을 위해 윈도우 환경에 무료로 사용할 수 있는 관계형 데이터베이스인 MySQL을 설치하는 방법을 알아보자. 우선 MySQL 공식 홈페이지(<https://www.mysql.com/>)에 접속한다.



그림 16-4 MySQL 공식 홈페이지

검색창에 ‘MySQL community download’를 입력하여 [MySQL Community Downloads] 페이지로 이동한다. 페이지 항목 중 [MySQL Installer for Windows]를 선택한다.

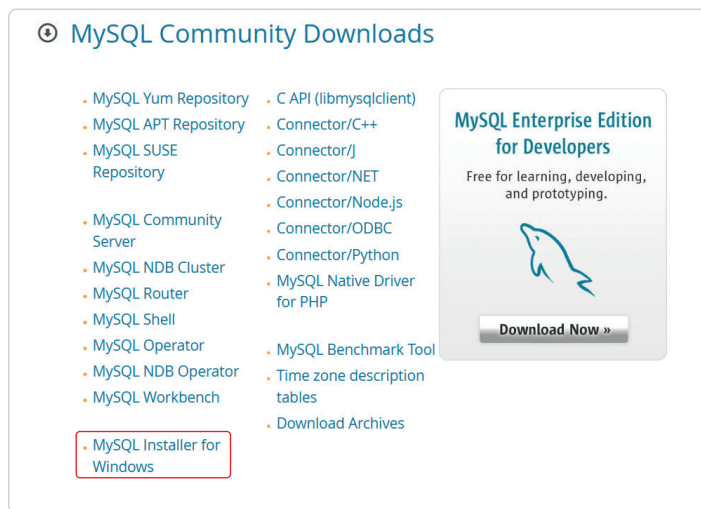


그림 16-5 [MySQL Community Downloads] 페이지

‘MySQL Installer 8.0.44’를 다운로드하는 페이지가 나타나면, 두 번째 항목인 ‘Windows (x86, 32-bit), MSI Installer’의 <Download>를 클릭한다. 로그인/가입 요청 화면이 나오면 보통 하단의 ‘No thanks, just start my download.’를 클릭하여 건너뛸 수 있다.

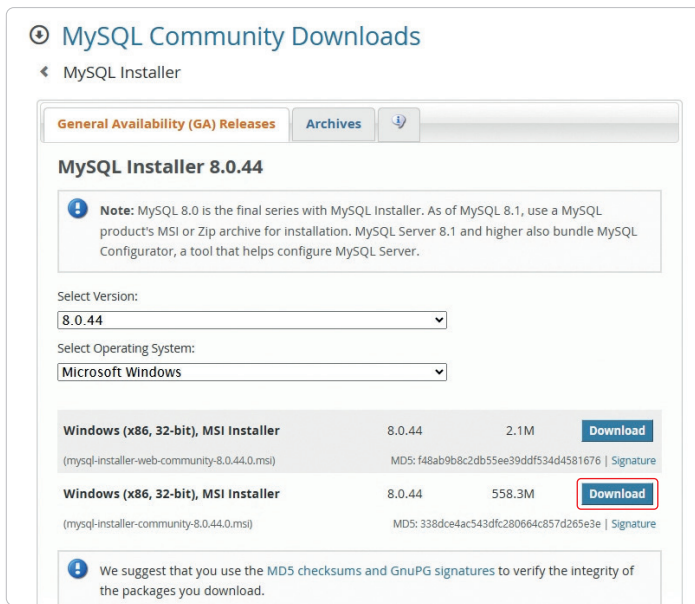


그림 16-6 [MySQL Installer for Windows] 페이지

2. MySQL 설치

다운로드한 MSI 파일을 더블클릭하여 설치를 시작한다. 다음과 같이 설치 유형을 선택하는 [Choosing a Setup Type] 페이지가 나타나면, 일반적인 개발 환경에는 필요한 구성 요소만 선택할 수 있는 ‘Custom’을 선택하고 <Next>를 클릭한다.

TIP ‘Custom’이 아닌 ‘Developer Default’를 선택해도 무방하다.

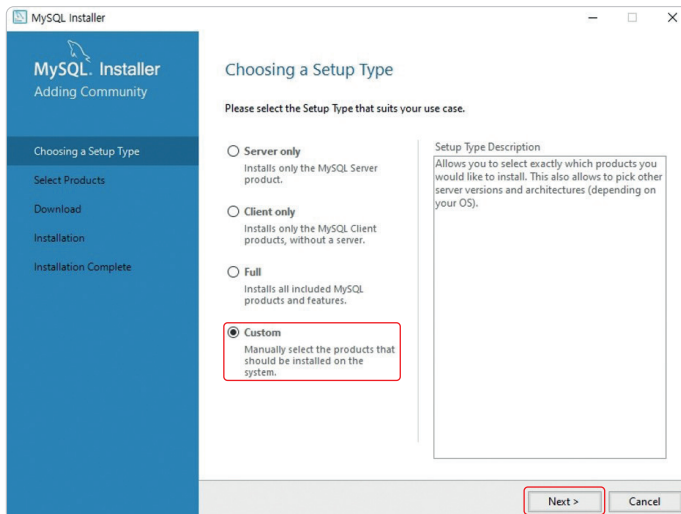


그림 16-7 [설치 유형 선택(Choosing a Setup Type)] 페이지

‘Custom’을 선택했다면, 다음 세 가지 핵심 구성 요소를 [Products To Be Installed:] 목록으로 옮긴 후 <Next>를 클릭한다.

- MySQL Server(데이터베이스 엔진 본체)
- MySQL Workbench(GUI 기반 관리 및 개발 도구)
- MySQL Shell(명령줄 인터페이스 툴, 선택 사항)

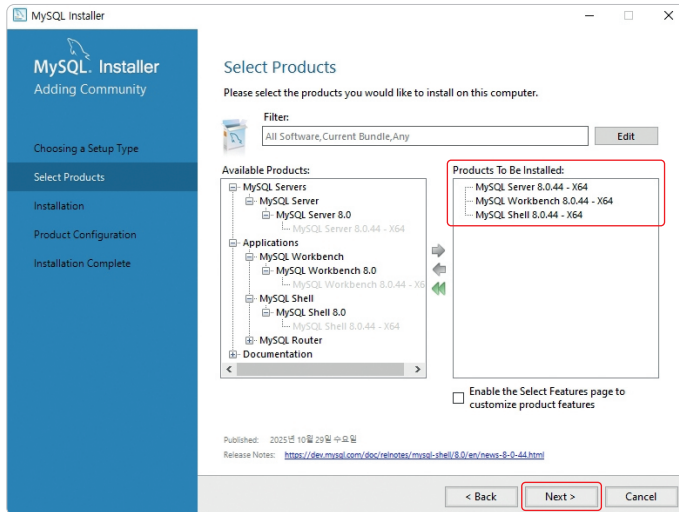


그림 16-8 [제품 선택(Select Products)] 페이지

선택한 제품 목록을 확인한 뒤 <Execute>를 클릭해 설치하고, 완료될 때까지 기다린다.

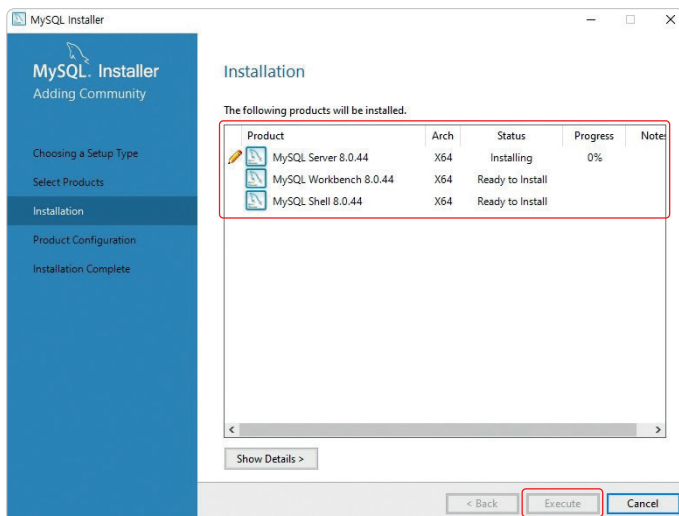


그림 16-9 [설치(Installation)] 페이지

3. MySQL 환경 설정

설치가 완료되면 서버 환경 설정 Configuration 이 자동으로 시작된다. 'Config Type'은 'Development Computer'를 선택하고, 'Port' 번호는 기본값인 3306을 그대로 사용한다. JDBC 연결 시 이 포트 번호를 사용하게 된다. 이후 <Next>를 클릭한다.

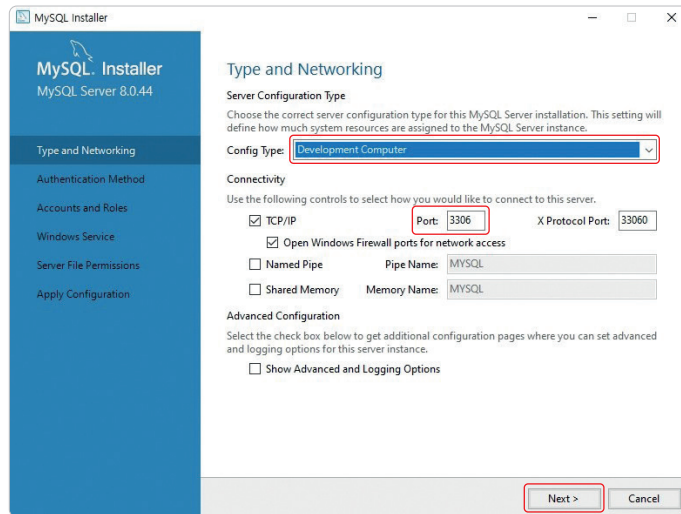


그림 16-10 [서버 환경 설정(Type and Networking)] 페이지

인증 방법을 선택하는 [Authentication Method] 페이지에서 'Use Strong Password Encryption for Authentication(RECOMMENDED)'를 선택하고 <Next>를 클릭한다.

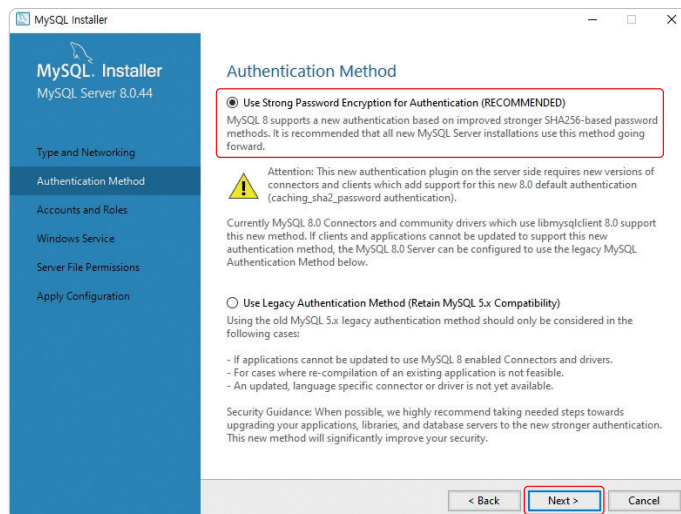


그림 16-11 [인증 방법 선택(Authentication Method)] 페이지

계정 및 역할을 설정하는 [Accounts and Roles] 페이지에서 ‘User Name’과 ‘Root Password’를 설정한다. JDBC 연결시에 사용할 가장 중요한 비밀번호이므로 안전한 비밀번호를 입력하고 기억해야 한다. 이후 <Next>를 클릭한다.

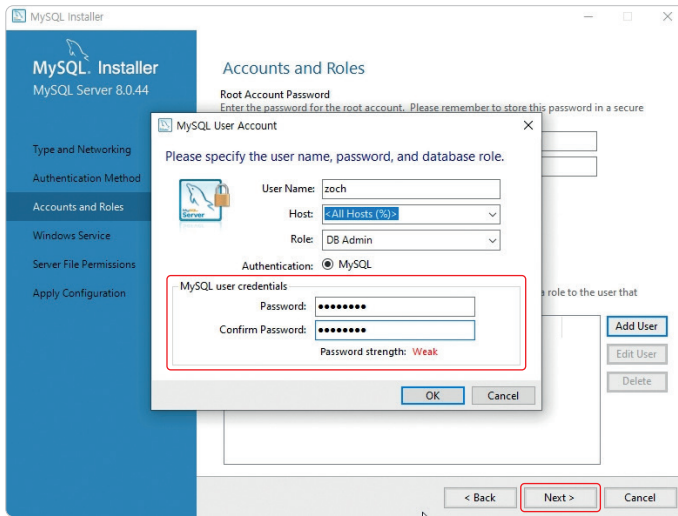


그림 16-12 [계정 및 역할 설정(Accounts and Roles)] 페이지

[Windows Service] 페이지에서 MySQL 서버를 윈도우 서비스로 등록하여 자동으로 시작되도록 설정한다. 기본값인 ‘Start the MySQL Server at System Startup’ 옵션을 유지하고 <Next>를 클릭한다.

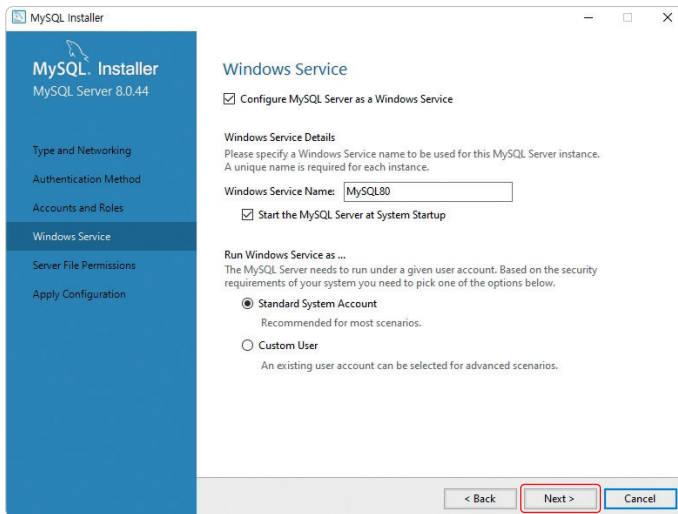


그림 16-13 [윈도우 서비스(Windows Service)] 페이지

[Apply Configuration] 페이지에서 <Execute>를 클릭하여 모든 설정 사항을 서버에 적용한다. 모든 항목에 초록색 체크 표시가 뜨면 <Finish>를 클릭하여 MySQL 설치를 마친다.

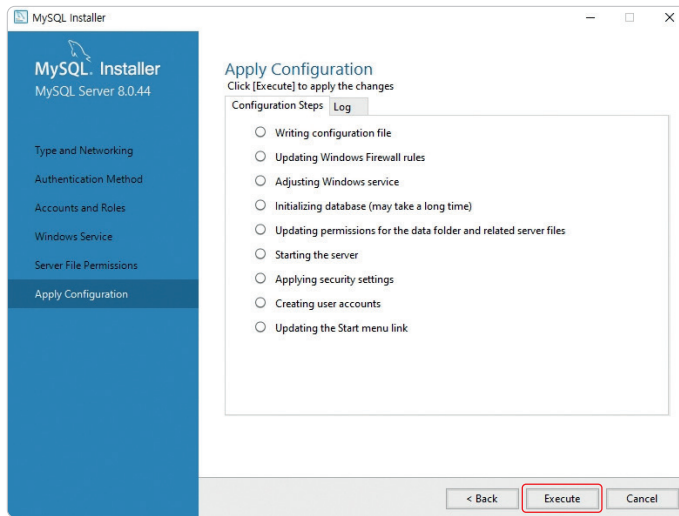


그림 16-14 [설정 적용(Apply Configuration)] 페이지

4. MySQL 워크벤치 실행

정상적으로 설치가 끝나면 MySQL 워크벤치를 실행한다. ❶ [Local Instance (root@127.0.0.1:3306)] 연결을 클릭하고, 3단계에서 설정한 ❷ 'Root Password'를 입력한 뒤 ❸ <OK>를 클릭하여 접속한다.

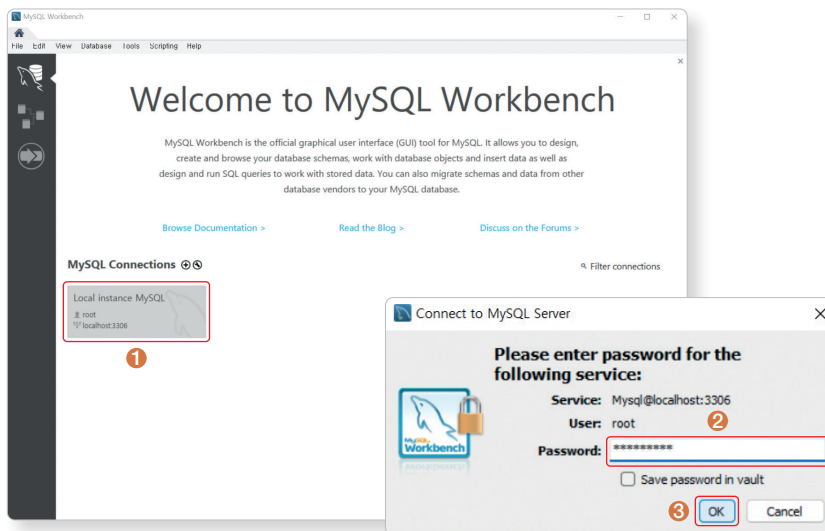


그림 16-15 MySQL 워크벤치 접속 화면

정상적으로 워크벤치 화면이 나타나면 MySQL 서버 설치가 성공적으로 완료된 것이다.

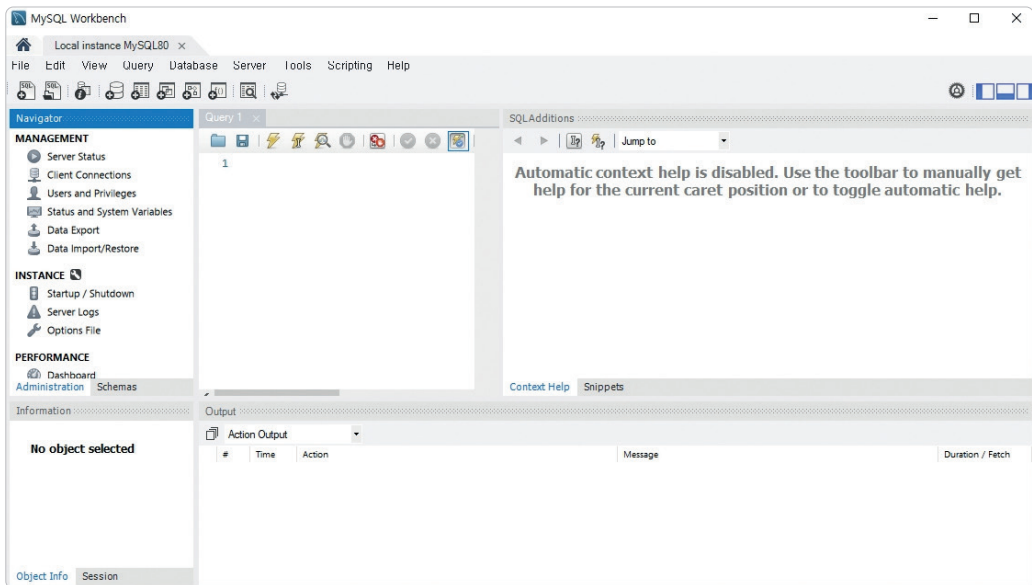


그림 16-16 MySQL 워크벤치 실행 화면

3 데이터베이스 및 테이블 생성

2절 ‘JDBC 프로그래밍’에서 실습할 JDBC 테스트를 위해 MySQL에 간단한 데이터베이스와 테이블을 미리 생성해보자. MySQL 워크벤치나 ‘Command Line Client’를 사용하여 다음 순서를 따라 데이터베이스와 테이블을 생성한다.

- 1 jdbc_test_db라는 이름을 가진 데이터베이스를 생성하는 SQL 명령은 다음과 같다.

```
CREATE DATABASE jdbc_test_db;
```

- 2 jdbc_test_db 데이터베이스가 생성되면 다음과 같이 사용 설정을 한다.

```
USE jdbc_test_db;
```


③ 이제 다음 SQL 명령을 사용하여 테이블을 생성한다.

```
CREATE TABLE employees (  
    id INT PRIMARY KEY AUTO_INCREMENT,  
    name VARCHAR(100) NOT NULL,  
    department VARCHAR(50),  
    salary INT  
);
```

④ 생성한 테이블에 다음과 같은 명령을 사용하여 샘플 데이터를 삽입한다. 다음 SQL 문은 employees (name, department, salary) 테이블에 값을 삽입(INSERT)한다.

```
INSERT INTO employees (name, department, salary) VALUES ('김철수', '개발', 5000);  
INSERT INTO employees (name, department, salary) VALUES ('이영희', '인사', 4500);  
INSERT INTO employees (name, department, salary) VALUES ('박민준', '영업', 4800);
```

①~④까지의 과정을 마쳤다면, 샘플 데이터가 정확하게 삽입되었는지 확인하기 위하여 다음 SQL 명령을 실행한다. 다음 SQL 문은 employees 테이블의 모든 데이터(*)를 가져온다(SELECT).

```
SELECT * FROM employees;
```

SQL 명령문을 사용하여 워크벤치에 생성된 데이터베이스와 테이블 결과 화면은 [그림 16-17]과 같다.

	id	name	department	salary
▶	1	김철수	개발	5000
	2	이영희	인사	4500
	3	박민준	영업	4800
*	NULL	NULL	NULL	NULL

그림 16-17 MySQL 워크벤치의 결과 화면

생성형 AI와 공부하기

SQL에 사용되는 주요 명령어를 설명하고, 간단한 예제를 작성해서 활용 방법을 설명해줘.

1 JDBC와 드라이버

1. JDBC의 이해

JDBC(Java DataBase Connectivity)는 자바 언어로 작성된 애플리케이션이 다양한 관계형 데이터베이스 관리 시스템에 접속하고, SQL 쿼리를 실행하며, 결과를 처리할 수 있도록 표준화된 방법을 제공하는 자바 API이다. JDBC는 주로 `java.sql` 및 `javax.sql` 패키지에 정의된 인터페이스와 클래스로 구성되며, 가장 핵심적인 요소는 다음과 같다.

- **DriverManager(클래스)**: 다양한 JDBC 드라이버를 관리하고, 데이터베이스 연결을 설정하는 역할을 한다. 응용 프로그램이 데이터베이스 연결을 요청하면 적절한 드라이버를 찾아 `Connection` 객체를 반환한다.
- **Driver(인터페이스)**: 각 데이터베이스가 JDBC API 정의에 따라 구현해야 하는 인터페이스로, 실제 데이터베이스와의 통신을 담당한다.
- **Connection(인터페이스)**: 특정 데이터베이스와의 연결(세션)을 나타낸다. SQL 명령을 실행하기 위한 `Statement` 객체를 생성하는 데 사용되며, 트랜잭션 관리 기능도 제공한다.
- **Statement(인터페이스)**: SQL 쿼리를 데이터베이스에 전송하고 실행하는 데 사용하는 객체이다.
- **SQLException(클래스)**: JDBC 작업 중 발생하는 데이터베이스 관련 오류를 처리한다.

JDBC를 사용하여 데이터베이스에 접근하고 SQL을 실행하는 표준적인 단계는 다음과 같다.

- ① **JDBC 드라이버 로딩**: 사용할 데이터베이스에 맞는 JDBC 드라이버 클래스를 로드한다.
- ② **Connection 객체 생성**: `DriverManager.getConnection(url, user, password)` 메서드를 사용하여 데이터베이스에 연결하고 `Connection` 객체를 얻는다. URL은 데이터베이스의 위치와 유형을 지정한다.
- ③ **Statement 객체 생성**: `Connection` 객체를 사용하여 `Statement` 또는 `PreparedStatement` 객체를 생성한다.
- ④ **SQL 쿼리 실행**: `Statement` 객체의 메서드를 사용하여 SQL 쿼리를 실행한다.

- ⑤ **ResultSet 객체로 데이터 조회/처리** : SELECT 쿼리 실행 결과로 반환된 ResultSet 객체로부터 next()를 호출하여 행을 순회하고, getString(), getInt() 등의 메서드를 사용하여 컬럼 값을 가져와 처리한다.
- ⑥ **자원 해제** : 사용한 ResultSet, Statement, Connection 객체를 close() 메서드를 호출하여 역순으로 닫는다. 이는 메모리 누수를 방지하고, 데이터베이스의 연결을 해제하는 중요한 과정이다.

데이터베이스에 접속하려면 몇 가지 핵심 정보가 필요하다. 먼저 DBMS 벤더가 제공하는 JDBC 드라이버 클래스 이름이 있어야 하고, 데이터베이스의 위치와 이름을 포함한 JDBC URL, 접근 가능한 사용자 이름과 비밀번호가 필요하다. 이외에도 연결 시간 제한, 문자셋 설정, SSL 옵션 같은 부가 설정이 존재한다.

2. JDBC 드라이버 설치

자바를 사용하여 데이터베이스에 접근하려면 MySQL의 JDBC 드라이버인 ‘Connector/J’가 필요하다. 이를 위해 MySQL 공식 사이트의 검색창에서 ‘mysql connector j download’를 검색한다. 다운로드 페이지 목록에서 ‘Connector/J’ 항목을 찾아서 클릭한다.

‘Connector/J’ 다운로드 페이지의 [Select Operating System:](운영체제 선택) 항목에서 [Platform Independent](플랫폼 독립적)를 선택한다. ‘Connector/J’는 순수한 Java JAR 파일이므로, 윈도우, 맥, 리눅스 등 특정 운영체제에 종속되지 않고 사용할 수 있다.

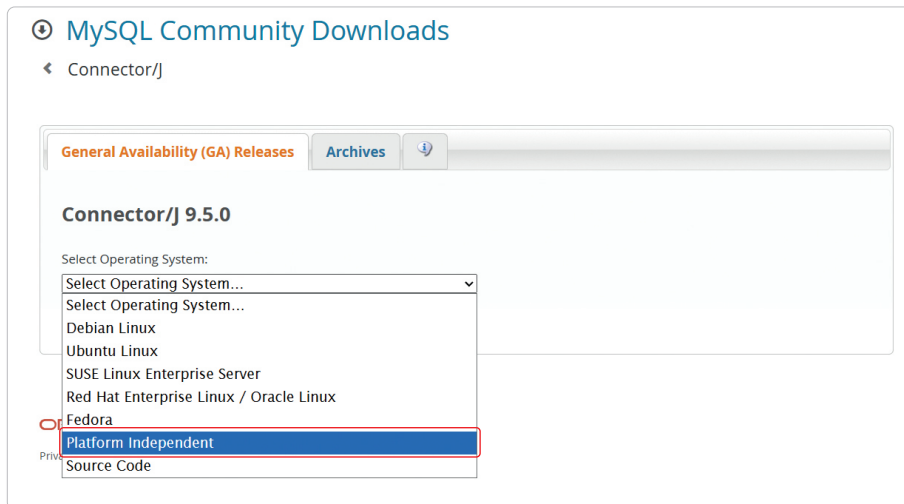


그림 16-18 Connector/J의 운영체제 선택 페이지

‘Connector/J’ 다운로드 페이지에는 보통 두 가지 압축 파일 형식이 있다. 두 번째 항목인 .zip 파일을 선택한 후, 옆에 있는 <Download>를 클릭한다.



그림 16-19 Connector/J 다운로드 페이지

다운로드가 완료되면, 압축을 풀고 ‘mysql-connector-j-x.x.x.jar’(또는 이전 버전의 경우 mysql-connector-java-x.x.x.jar) 형태의 파일을 찾는다. 이 .jar 파일이 바로 이클립스 프로젝트의 빌드 경로 Build Path에 추가해야 하는 JDBC 드라이버이다. 이 파일을 적당한 폴더에 저장한 후 이클립스에서 JDBC 드라이버를 등록해보자.

먼저 프로젝트를 우클릭하고 [Properties]를 선택한다. 이때, 제공되는 예제 파일에서는 ‘chap16’을 선택한다. 좌측 메뉴에서 ❶ [Java Build Path]를 선택한 후, ❷ [Libraries] 탭으로 이동한다. [Classpath]를 클릭하면 오른쪽 메뉴가 활성화된다. 메뉴 중 ❸ [Add External JARs...]를 선택한다. 이후 다운로드한 mysql-connector-j-x.x.x.jar 파일의 경로를 찾아 추가한다.

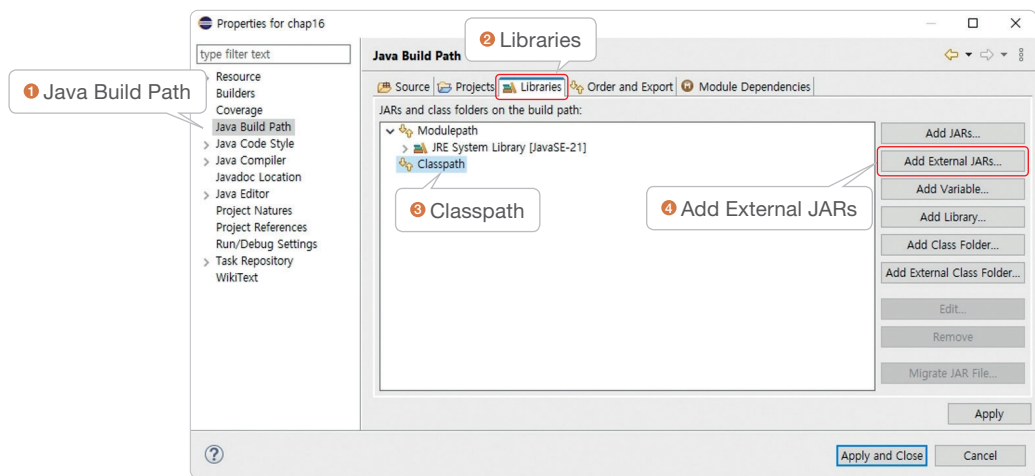


그림 16-20 JDBC 드라이버 등록하기

제공되는 예제 파일의 'chap16' 폴더 안에 'mysql-connector-j-9.5.0.jar'가 저장되어 있으니 이를 립스에서 해당 폴더가 정확하게 등록되어 있는지를 확인한다.

2 JDBC의 사용

1. 데이터베이스 접근 순서

이제 JDBC를 사용하여 데이터베이스를 조작해보자. 앞서 사용한 `jdbc_test_db` 데이터베이스의 `employees` 테이블을 이어지는 실습에서 계속 사용할 것이다. JDBC를 사용하여 데이터베이스에 접근하는 방법은 다음과 같은 순서로 진행된다.

- 1 다음과 같은 명령을 사용하여 드라이버를 로드한다. 다음 코드는 `DriverManager`가 해당 드라이버를 사용하여 연결을 설정할 수 있도록 준비하는 과정이다. JDBC 4.0 이후 버전에서는 이 과정이 선택 사항이 되었지만, 명시적으로 작성하는 경우도 많다.

```
Class.forName("com.mysql.cj.jdbc.Driver")
```

- 2 URL, 사용자 이름, 비밀번호를 이용해 DBMS에 연결한다. 여기서 `password`는 실제 사용하는 비밀번호를 작성한다.

```
Connection conn = DriverManager.getConnection("jdbc:mysql://localhost:3306/mydb",  
"root", "password");
```

- 3 SQL 문을 작성하고 `Statement`나 `PreparedStatement` 객체를 이용해 DBMS에 전달한다.

```
String sql = "INSERT INTO employees (name, department, salary) VALUES (?, ?, ?)";  
PreparedStatement pstmt = conn.prepareStatement(sql);
```

- 4 DBMS가 반환한 데이터를 `pstmt` 객체로 받아와 자바 코드에서 처리하고, 사용이 끝난 객체를 닫아 연결을 종료한다.

이 과정을 통해 자바 프로그램은 데이터베이스와 안전하게 데이터를 주고받을 수 있다.

2. 데이터 삽입

[예제 16-1]은 데이터베이스에 새로운 직원을 추가하는 코드이다.

예제 16-1

데이터 삽입하기

JdbcInsertExample.java

```
01 package sec02.exam01;
02
03 import java.sql.Connection;
04 import java.sql.DriverManager;
05 import java.sql.PreparedStatement;
06 import java.sql.SQLException;
07
08 public class JdbcInsertExample {
09     private static final String DB_URL = "jdbc:mysql://localhost:3306/jdbc_test_db";
10     private static final String USER = "root";
11     private static final String PASS = "password"; // 실제 루트 패스워드 사용!
12
13     public static void insertData(String name, String department, int salary) {
14         String sql = "INSERT INTO employees (name, department, salary) VALUES
15             (?, ?, ?)";
16
17         try (Connection conn = DriverManager.getConnection(DB_URL, USER, PASS);
18             PreparedStatement pstmt = conn.prepareStatement(sql)) {
19
20             // 1. 플레이스 홀더에 값 설정, JDBC의 인덱스는 1부터 시작
21             pstmt.setString(1, name);
22             pstmt.setString(2, department);
23             pstmt.setInt(3, salary);
24
25             // 2. 쿼리 실행
26             int rows = pstmt.executeUpdate();
27
28             System.out.println(name + " 삽입 성공: " + rows + "개의 행이 삽입됨");
29
30         } catch (SQLException e) {
31             System.err.println("삽입 중 SQL 오류: " + e.getMessage());
32             e.printStackTrace();
33         }
34     }
35
36     public static void main(String[] args) {
37         try {
```

```

37         Class.forName("com.mysql.cj.jdbc.Driver");
38     } catch (ClassNotFoundException e) {
39         System.err.println("MySQL JDBC 드라이버를 찾을 수 없음!");
40         e.printStackTrace();
41         return;
42     }
43
44     System.out.println("--- 새로운 직원 데이터 삽입 시작 ---");
45     insertData("정민지", "회계", 5200);
46     insertData("최현우", "마케팅", 4900);
47     insertData("강하늘", null, 4000);
48     System.out.println("--- 삽입 작업 완료 ---");
49 }
50 }

```

```

--- 새로운 직원 데이터 삽입 시작 ---
정민지 삽입 성공: 1개의 행이 삽입됨
최현우 삽입 성공: 1개의 행이 삽입됨
강하늘 삽입 성공: 1개의 행이 삽입됨
--- 삽입 작업 완료 ---

```

3~6행은 JDBC 작업을 위해 필요한 핵심 클래스를 가져온다.

9행 `DB_URL`은 데이터베이스에 접속하기 위한 JDBC URL이다. `jdbc:mysql://`은 MySQL JDBC 드라이버를 사용함을 나타내며, `localhost:3306`은 데이터베이스 서버의 위치와 포트(기본 포트)를 나타낸다. `jdbc_test_db`는 접속할 데이터베이스의 이름이다.

10행 `USER`는 데이터베이스 접속 사용자의 이름이며, 11행 `PASS`는 데이터베이스 접속 비밀번호이다. 실제로 사용하는 비밀번호로 변경해야 한다.

14행의 `?`는 플레이스 홀더 place holder로, 실제 값이 실행 시점에 바인딩(할당)될 위치를 나타낸다.

16~33행에서는 `try-with-resources` 구문을 사용하여 `Connection`과 `PreparedStatement`가 자동으로 닫히도록 한다.

16행 `DriverManager.getConnection()`은 데이터베이스 연결을 설정하고 `Connection` 객체를 반환하고, 17행 `conn.prepareStatement(sql)`은 정의된 SQL 쿼리를 기반으로 `PreparedStatement` 객체를 생성한다.

20~22행은 `setString()`, `setInt()` 등의 메서드를 사용하여 쿼리의 ? 위치에 실제 매개변수 값을 안전하게 바인딩한다. JDBC 인덱스는 0이 아니라 1부터 시작하는 것에 주의해야 한다.

25행 `executeUpdate()` 메서드는 `INSERT`, `UPDATE`, `DELETE`와 같은 데이터를 변경하는 쿼리를 실행한다. 실행 결과로 영향을 받은 행의 개수를 정수(rows)로 반환한다.

37행에서는 `Class.forName()`을 사용하여 MySQL JDBC 드라이버 클래스를 메모리에 로드한다.

38행 `ClassNotFoundException`은 드라이버 클래스 파일(일반적으로 .jar 파일)이 프로젝트에 포함되어 있지 않거나 경로가 잘못된 경우 발생하는 예외를 처리한다.

45~47행은 정의된 `insertData` 메서드를 호출하여 세 건의 직원 데이터를 데이터베이스에 삽입한다.

[예제 16-1]이 실행되고 난 이후에 MySQL 워크벤치에서 `SELECT * FROM employees;`를 사용하여 테이블의 상태를 출력하면 [그림 16-21]과 같다.

	id	name	department	salary
▶	1	김철수	개발	5000
	2	이영희	인사	4500
	3	박민준	영업	4800
	4	정민지	회계	5200
	5	최현우	마케팅	4900
	6	강하늘	NULL	4000
*	NULL	NULL	NULL	NULL

그림 16-21 MySQL 워크벤치의 결과 화면

3. 데이터 조회

[예제 16-2]는 데이터베이스에서 직원의 이름과 연봉을 출력하는 코드이다.

예제 16-2 데이터 조회하기

JdbcSelectExample.java

```
01 package sec02.exam02;
02
03 import java.sql.Connection;
04 import java.sql.DriverManager;
05 import java.sql.PreparedStatement;
06 import java.sql.ResultSet;           // ResultSet 클래스 импорт
07 import java.sql.SQLException;
08
09 public class JdbcSelectExample {
```



```

10     private static final String DB_URL = "jdbc:mysql://localhost:3306/jdbc_
test_db";
11     private static final String USER = "root";
12     private static final String PASS = "password";    // 실제 루트 패스워드 사용!
13
14     public static void selectEmployeeData() {
15         String sql = "SELECT name, salary FROM employees";
16
17         try (Connection conn = DriverManager.getConnection(DB_URL, USER, PASS);
18             PreparedStatement pstmt = conn.prepareStatement(sql);
19             ResultSet rs = pstmt.executeQuery()) {    // 실행 및 결과 저장
20
21             System.out.println("-----");
22             System.out.println("직원 이름 및 연봉 목록");
23             System.out.println("-----");
24
25             while (rs.next()) {
26                 String name = rs.getString("name");
27                 int salary = rs.getInt("salary");
28
29                 System.out.printf("이름: %-8s | 연봉: %d만원\n", name, salary);
30             }
31             System.out.println("-----");
32
33             } catch (SQLException e) {
34                 System.err.println("조회 중 SQL 오류: " + e.getMessage());
35                 e.printStackTrace();
36             }
37     }
38
39     public static void main(String[] args) {
40         try {
41             Class.forName("com.mysql.cj.jdbc.Driver");
42         } catch (ClassNotFoundException e) {
43             System.err.println("MySQL JDBC 드라이버를 찾을 수 없음!");
44             e.printStackTrace();
45             return;
46         }
47
48         System.out.println("--- 직원 데이터 조회 시작 ---");
49         selectEmployeeData();                                // 조회 메서드 호출

```

```

50         System.out.println("--- 조회 작업 완료 ---");
51     }
52 }

```

--- 직원 데이터 조회 시작 ---

 직원 이름 및 연봉 목록

이름: 김철수	연봉: 5000만원
이름: 이영희	연봉: 4500만원
이름: 박민준	연봉: 4800만원
이름: 정민지	연봉: 5200만원
이름: 최현우	연봉: 4900만원
이름: 강하늘	연봉: 4000만원

 --- 조회 작업 완료 ---

6행에서 `java.sql.ResultSet` 클래스가 추가되었다. 이 클래스는 `SELECT` 쿼리를 실행한 후, 데이터베이스로부터 반환되는 테이블 형태의 결과 집합을 담아 자바 코드에서 접근할 수 있도록 하는 객체이다.

15행 `SELECT name, salary FROM employees`는 데이터를 조회하기 위한 SQL 문이다.

25~30행은 `ResultSet` 객체에서 데이터를 실제로 읽어와 출력하는 반복문이다. `while (rs.next())`는 `ResultSet`의 커서를 데이터가 담긴 다음 행으로 이동시키고, 더 이상 읽을 행이 없을 때까지 루프를 반복한다. 반복문 내부에서는 `rs.getString("name")` 또는 `rs.getInt("salary")`와 같은 메서드를 사용하여 현재 커서가 가리키는 행의 특정 칼럼(이름으로 지정)에 저장된 데이터를 자바 변수로 추출한다.

4. 데이터 삭제

[예제 16-3]은 데이터베이스에서 ID 6번 ‘강하늘’의 데이터를 삭제하는 코드이다.

예제 16-3 데이터 삭제하기

JdbcDeleteExample.java

```

01 package sec02.exam03;
02
03 import java.sql.Connection;

```

```

04 import java.sql.DriverManager;
05 import java.sql.PreparedStatement;
06 import java.sql.SQLException;
07
08 public class JdbcDeleteExample {
09     private static final String DB_URL = "jdbc:mysql://localhost:3306/jdbc_
test_db";
10     private static final String USER = "root";
11     private static final String PASS = "password"; // 실제 루트 패스워드 사용!
12
13     public static void deleteData(int id) {
14         String sql = "DELETE FROM employees WHERE id = ?";
15
16         try (Connection conn = DriverManager.getConnection(DB_URL, USER, PASS);
17             PreparedStatement pstmt = conn.prepareStatement(sql);) {
18
19             pstmt.setInt(1, id);
20             int rows = pstmt.executeUpdate();
21
22             if (rows > 0) {
23                 System.out.println("ID " + id + " 직원 삭제 성공: " + rows +
"개의 행이 삭제됨");
24             } else {
25                 System.out.println("ID " + id + " 직원 삭제 실패: 해당 ID를 가진
행이 존재하지 않음");
26             }
27
28         } catch (SQLException e) {
29             System.err.println("삭제 중 SQL 오류: " + e.getMessage());
30             e.printStackTrace();
31         }
32     }
33
34     public static void main(String[] args) {
35         try {
36             Class.forName("com.mysql.cj.jdbc.Driver");
37         } catch (ClassNotFoundException e) {
38             System.err.println("MySQL JDBC 드라이버를 찾을 수 없음!");
39             e.printStackTrace();
40             return;
41         }
42

```

```

43         final int ID_TO_DELETE = 6;
44
45         System.out.println("ID " + ID_TO_DELETE + " 직원 데이터 삭제 시작 ");
46         deleteData(ID_TO_DELETE);           // ID 6번 직원 삭제 호출
47         System.out.println("--- 삭제 작업 완료 ---");
48     }
49 }

```

```

ID 6 직원 데이터 삭제 시작
ID 6 직원 삭제 성공: 1개의 행이 삭제됨
--- 삭제 작업 완료 ---

```

14행 `DELETE FROM employees WHERE id = ?`는 데이터를 삭제하는 SQL 문이다. 여기서 `WHERE id = ?`는 테이블 전체가 아닌 `id`의 값이 플레이스 홀더(?)에 바인딩된 값과 일치하는 행만을 삭제하도록 제한하는 역할을 한다. 만약 `WHERE` 절이 생략되면 테이블의 모든 데이터가 삭제되는 위험한 상황이 발생할 수 있다.

19행 `pstmt.setInt(1, id)`는 코드를 통해 `deleteData` 메서드가 받은 `id` 값을 SQL 쿼리의 첫 번째 플레이스 홀더(?)에 바인딩한다.

20행 `int rows = pstmt.executeUpdate()`는 실제 삭제 쿼리를 데이터베이스에 전달하고 실행한 후, 그 결과로 실제로 삭제된 행의 개수(`rows`)를 반환한다.

22행은 삭제된 행의 개수(`rows`)가 0보다 큰지 확인하여 삭제 성공 여부를 출력한다.

[예제 16-3]이 실행되고 난 이후에 MySQL 워크벤치에서 `SELECT * FROM employees;`를 사용하여 테이블의 상태를 출력하면 [그림 16-22]와 같다.

	id	name	department	salary
▶	1	김철수	개발	5000
	2	이영희	인사	4500
	3	박민준	영업	4800
	4	정민지	회계	5200
	5	최현우	마케팅	4900
*	NULL	NULL	NULL	NULL

그림 16-22 MySQL 워크벤치의 결과 화면

5. 데이터 변경

[예제 16-4]는 데이터베이스에서 ID 1번 ‘김철수’의 부서를 마케팅으로 변경하는 코드이다.

예제 16-4

데이터 변경하기

JdbcUpdateExample.java

```
01 package sec02.exam04;
02
03 import java.sql.Connection;
04 import java.sql.DriverManager;
05 import java.sql.PreparedStatement;
06 import java.sql.SQLException;
07
08 public class JdbcUpdateExample {
09     private static final String DB_URL = "jdbc:mysql://localhost:3306/jdbc_
test_db";
10     private static final String USER = "root";
11     private static final String PASS = "password"; // 실제 루트 패스워드 사용!
12
13     public static void updateDepartment(int id, String newDepartment) {
14         String sql = "UPDATE employees SET department = ? WHERE id = ?";
15
16         try (Connection conn = DriverManager.getConnection(DB_URL, USER, PASS);
17             PreparedStatement pstmt = conn.prepareStatement(sql);) {
18
19             pstmt.setString(1, newDepartment);
20             pstmt.setInt(2, id);
21
22             int rows = pstmt.executeUpdate();
23
24             if (rows > 0) {
25                 System.out.println("ID " + id + " 직원 부서 변경 성공:" +
newDepartment + " (총 " + rows + "개 행 업데이트)");
26             } else {
27                 System.out.println("ID " + id + " 직원 부서 변경 실패: 해당 ID를
가진 행이 존재하지 않음");
28             }
29
30         } catch (SQLException e) {
31             System.err.println("업데이트 중 SQL 오류: " + e.getMessage());
32             e.printStackTrace();
33         }
34     }
```

```

35
36     public static void main(String[] args) {
37         try {
38             Class.forName("com.mysql.cj.jdbc.Driver");
39         } catch (ClassNotFoundException e) {
40             System.err.println("MySQL JDBC 드라이버를 찾을 수 없음!");
41             e.printStackTrace();
42             return;
43         }
44
45         final int TARGET_ID = 1;
46         final String NEW_DEPARTMENT = "마케팅";
47
48         System.out.println("ID " + TARGET_ID + " 직원 데이터 업데이트 시작 (부서: "
+ NEW_DEPARTMENT + ") ---");
49         updateDepartment(TARGET_ID, NEW_DEPARTMENT);
50         System.out.println("--- 업데이트 작업 완료 ---");
51     }
52 }

```

```

ID 1 직원 데이터 업데이트 시작 (부서: 마케팅) ---
ID 1 직원 부서 변경 성공:마케팅 (총 1개 행 업데이트)
--- 업데이트 작업 완료 ---

```

14행 `UPDATE employees`는 `employees` 테이블의 데이터를 수정하겠다는 명령이며, `SET department = ?`는 `department`의 값을 첫 번째 플레이스 홀더(?)에 바인딩되는 새로운 값으로 설정하겠다는 의미이다. `WHERE id = ?`는 업데이트 대상이 두 번째 플레이스 홀더(?) 값과 일치하는 ID를 가진 행 하나임을 명확히 지정하여, 실수로 테이블 전체가 업데이트되는 것을 방지하는 역할을 한다.

19행 `pstmt.setString(1, newDepartment)`는 쿼리의 `SET` 부분에 해당하는 새로운 값을 첫 번째 ?에 바인딩한다.

20행 `pstmt.setInt(2, id)`는 쿼리에서 `WHERE` 조건에 해당하는 대상의 ID를 두 번째 ?에 바인딩한다.

22행 `int rows = pstmt.executeUpdate();`는 업데이트 쿼리를 실행하며, 실행 후 실제로 업데이트된 행의 개수(`rows`)를 정숫값으로 반환한다.

24행에서 반환된 `rows` 값이 0보다 큰지를 확인하여 업데이트 성공 여부를 출력한다.

[예제 16-4]가 실행되고 난 이후에 MySQL 워크벤치에서 `SELECT * FROM employees;`를 사용하여 테이블의 상태를 출력하면 [그림 16-23]과 같다.

	id	name	department	salary
▶	1	김철수	마케팅	5000
	2	이영희	인사	4500
	3	박민준	영업	4800
	4	정민지	회계	5200
	5	최현우	마케팅	4900
*	NULL	NULL	NULL	NULL

그림 16-23 MySQL 워크벤치의 결과 화면

생성형 AI와 공부하기

jdbc_test_db 데이터베이스의 id, name, department, salary 열을 가진 employees 테이블에서 연봉이 50000이 넘는 사원의 이름만 출력하는 JDBC 코드를 만들고, 코드의 구성을 자세히 설명해줘.

- 01 데이터베이스 관리 시스템(DBMS)은 사용자와 데이터베이스를 연결하는 소프트웨어 시스템이며, 관계형 모델을 사용하는 현대의 DBMS를 관계 데이터베이스 관리 시스템(RDBMS)이라 부른다. 관계 모델에서 데이터는 테이블 형태로 구성되며, 구성된 테이블은 서로의 키를 중심으로 여러 관계를 형성한다.
- 02 데이터베이스에서 데이터는 이미 만들어진 틀에 저장되는데, 저장할 데이터의 구조, 데이터와 데이터 간의 관계, 제약조건을 정의한 것이 스키마이다. 스키마는 어떤 테이블이 있고, 각 테이블이 어떤 열을 가지며, 열의 타입과 제약조건은 무엇인지 등을 정의한다. 스키마는 데이터베이스의 설계도와 같아서, 처음에 스키마를 제대로 만들지 않으면 이후 데이터베이스를 사용할 때 문제가 발생할 수 있다.
- 03 데이터베이스에 저장된 데이터를 처리하는 언어가 SQL이다. SQL은 관계형 데이터베이스와 통신하기 위한 표준 언어이며, SQL을 사용하여 형식에 맞는 질문을 하면 DBMS가 응답한다.
- 04 JDBC는 자바 언어로 작성된 애플리케이션이 다양한 관계형 데이터베이스 관리 시스템에 접속하고, SQL 쿼리를 실행하며, 결과를 처리할 수 있도록 표준화된 방법을 제공하는 자바 API이다. JDBC는 주로 `java.sql` 및 `javax.sql` 패키지에 정의된 인터페이스와 클래스로 구성되며, 자바를 사용하여 데이터베이스에 접근하려면 JDBC 드라이버인 'Connector/J'가 필요하다.
- 05 JDBC를 사용하여 데이터베이스에 접근하는 방법은 다음과 같은 순서로 진행된다. 먼저 드라이버를 로드한 후 URL, 사용자 이름, 비밀번호를 이용해 DBMS에 연결한다. 이후 SQL 문을 작성하고 `Statement`나 `PreparedStatement` 객체를 이용해 DBMS에 전달하고, DBMS가 반환한 데이터를 `pstmt` 객체로 받아와 자바 코드에서 처리한다. 마지막으로 사용이 끝난 객체를 닫아 연결을 종료한다.

연습 문제

01 사용자와 데이터베이스를 연결하는 소프트웨어 시스템은 무엇인가?

- [illegible]

02 데이터베이스의 테이블이 가지는 열과 열의 타입, 제약조건을 정의한 개념을 찾으시오.

- ① JDBC
 - ② 스키마
 - ③ SQL
 - ④ RDBMS

03 ‘데이터베이스의 설계도’라는 의미와 가장 가까운 단어를 찾으시오.

- [illegible]

04 데이터베이스의 테이블에서 스키마 구조에 맞추어 저장되는 각 영역은 무엇인가?

- ① JDBC ② SQL
③ 필드 ④ 레코드

05 데이터베이스의 테이블에서 저장된 데이터 묶음 1개를 의미하는 단어를 찾으시오.

- [illegible]

06 데이터베이스에 저장된 데이터를 처리할 때 사용하는 언어를 찾으시오.

- [illegible]

12 SQL 명령어 중 조회에 사용하는 명령어를 찾으시오.

- | | |
|----------|----------|
| ① INSERT | ② SELECT |
| ③ DELETE | ④ UPDATE |

13 SQL 명령어 중 삭제에 사용하는 명령어를 찾으시오.

- | | |
|----------|----------|
| ① INSERT | ② SELECT |
| ③ DELETE | ④ UPDATE |

14 SQL 명령어 중 변경에 사용하는 명령어를 찾으시오.

- | | |
|----------|----------|
| ① INSERT | ② SELECT |
| ③ DELETE | ④ UPDATE |